



Politecnico
di Bari



Multimodal Search Engine

Technical Report

The Industrial Development Contract for Natuzzi S.p.A., in accordance with Article 9 of the Ministerial Decree for Economic Development dated December 9, 2014. 2Next: Industry 4.0 per il Design Italiano CUP: C83D16001570004

Principal Investigator:

Vito Walter Anelli, Ph.D. (vitowalter.aneli@poliba.it)

July 8, 2025

Table of Contents

1 Introduction	4
1.1 Problem Statement and Motivation	4
1.2 Research Objectives	5
1.3 Contributions	5
1.4 Report Organization	6
2 Background and Related Work	7
2.1 Information Retrieval Fundamentals	7
2.2 Multimodal Learning and Representation	9
2.3 Vector Databases and Similarity Search	12
3 System Architecture	14
3.1 Overall System Design	14
3.2 Multimodal Embedding Generation	17
3.3 Vector Database Integration	19
3.4 Search Engine Components	20
4 Implementation Details	24
4.1 Development Environment and Dependencies	24
4.2 Data Processing Pipeline	26
4.3 Model Configuration and Optimization	28
4.4 Database Operations	29
5 User Interfaces	32
5.1 Command-Line Interface	32
5.2 Web Interface with Gradio	36
5.3 Development Tools Integration	41
6 Discussion	42
6.1 Practical Applications	43
6.2 Future Improvements	44
7 Conclusion	47
7.1 Summary of Contributions	47
7.2 Future Research Directions	48
A - Code Documentation	49
A.1 Project Structure Overview	49
A.2 File Types	50
A.3 Installation and Setup	50
A.4 Dataset Download and Preparation	52
A.5 Embedding Generation and Database Population	52

Multimodal Search Engine

A.6	Running the Application	53
A.7	Development Workflow and VSCode Integration.....	55
A.8	Troubleshooting.....	55
A.9	References	56
Reference List		57

1 Introduction

1.1 Problem Statement and Motivation

The exponential growth of multimodal digital content has fundamentally transformed the landscape of information retrieval, creating unprecedented challenges in how we organize, search, and discover relevant information across heterogeneous data modalities. Traditional information retrieval systems, rooted in symbolic matching and statistical models such as TF-IDF and BM25, operate under the fundamental assumption of lexical overlap between queries and documents [24]. While these approaches have proven effective for text-centric applications, they are inherently inadequate for addressing the semantic gap that exists between textual descriptions and visual content, as well as the more nuanced challenge of cross-modal information needs.

The semantic gap—the disparity between low-level visual features and high-level semantic concepts—represents one of the most significant bottlenecks in modern retrieval systems. A user searching for "red leather chair with metal legs" may encounter a highly relevant product image that contains no textual metadata matching these specific terms, resulting in a retrieval failure despite perfect semantic alignment. This limitation becomes particularly pronounced in domains such as e-commerce, digital asset management, and content discovery platforms, where the inability to bridge visual and textual modalities directly impacts user satisfaction and business outcomes.

Recent advances in deep learning, particularly in vision-language modeling, have opened new avenues for addressing these challenges through dense, learned representations that capture semantic similarity across modalities. Models such as CLIP [26], BLIP [18], and their successors have demonstrated remarkable capabilities in learning joint embeddings that preserve semantic relationships between images and text in a shared vector space. However, the translation of these research breakthroughs into practical, scalable retrieval systems presents significant engineering and architectural challenges.

The computational complexity of neural embedding generation, the storage and indexing requirements for high-dimensional vectors, and the need for sub-linear search performance at scale necessitate sophisticated integration between state-of-the-art multimodal models and efficient vector database technologies. Furthermore, real-world deployment scenarios demand robust system architectures that can handle diverse query modalities—from pure text and image queries to complex multimodal compositions—while maintaining low latency and high relevance.

Contemporary vector databases such as Faiss, Milvus, and PostgreSQL with pgvector extension have emerged as critical infrastructure components for large-scale similarity search. However, their integration with multimodal retrieval systems remains an active area of research, particularly regarding optimal indexing strategies, similarity metrics, and the trade-offs between search quality and computational efficiency.

1.2 Research Objectives

This research addresses the aforementioned challenges through the design, implementation, and evaluation of a comprehensive multimodal search engine that seamlessly integrates state-of-the-art vision-language models with scalable vector database infrastructure. Our primary research objectives are articulated as follows:

Objective 1: Architectural Integration. Develop a modular, extensible architecture that effectively bridges the gap between multimodal representation learning and practical retrieval system deployment. This includes designing abstraction layers that enable seamless integration of different embedding models, vector database backends, and similarity computation strategies while maintaining system performance and scalability.

Objective 2: Embedding Space Optimization. Investigate and optimize the generation, storage, and retrieval of multimodal embeddings using BLIP2 [17] as the foundational vision-language model. This encompasses the analysis of embedding dimensionality trade-offs, normalization strategies, and the impact of different pre-training objectives on downstream retrieval performance.

Objective 3: Vector Database Performance. Systematically evaluate the performance characteristics of PostgreSQL with pgvector extension for multimodal retrieval workloads, including indexing strategy optimization, query performance analysis, and scalability assessment under varying dataset sizes and query patterns.

Objective 4: Cross-Modal Retrieval Quality. Establish comprehensive evaluation methodologies for assessing retrieval quality across different query modalities (text-to-image, image-to-text, image-to-image) and develop strategies for optimizing relevance while maintaining computational efficiency.

Objective 5: System Usability and Deployment. Design and implement user-friendly interfaces that demonstrate the practical applicability of the system, including both programmatic APIs and interactive web-based interfaces that enable real-world usage scenarios.

1.3 Contributions

This work makes several significant contributions to the intersection of multimodal learning, information retrieval, and systems engineering:

Technical Contributions:

1. **Unified Multimodal Search Architecture:** We present a novel system architecture that integrates BLIP2-based embedding generation with PostgreSQL/pgvector for scalable similarity search, providing a complete end-to-end solution for multimodal retrieval that bridges the gap between research models and production systems.
2. **Optimized Embedding Pipeline:** Our implementation includes carefully engineered preprocessing, encoding, and storage pipelines that maximize the semantic quality of learned representations while minimizing computational overhead. This includes adaptive batch processing, memory-efficient embedding generation, and optimized

vector normalization strategies.

3. **Hybrid Query Processing:** The system supports flexible query composition, enabling users to search using text, images, or combinations thereof, with automatic query type detection and appropriate embedding space mapping. This includes novel approaches to query-result similarity computation that account for multimodal query semantics.
4. **Database Schema Innovation:** We introduce a specialized database schema design that efficiently manages product hierarchies, variant relationships, and multimodal embeddings while supporting complex similarity queries with metadata filtering capabilities.

Practical Contributions:

1. **Production-Ready Architecture:** Our implementation addresses real-world deployment considerations including error handling, resource management, concurrent access patterns, and system monitoring capabilities.
2. **User Interface Design:** Both command-line and web-based interfaces demonstrate practical usage scenarios and provide templates for developing domain-specific applications.

1.4 Report Organization

This technical report is structured to provide a comprehensive treatment of both theoretical foundations and practical implementation details. The organization is designed to serve multiple audiences, from researchers interested in multimodal retrieval methodologies to practitioners seeking to implement similar systems.

Section 2 establishes the theoretical foundation by reviewing relevant work in information retrieval, multimodal learning, and vector database technologies. This section synthesizes current state-of-the-art approaches and identifies the specific research gaps that our work addresses.

Section 3 presents the overall system design, detailing the architectural decisions that enable effective integration of multimodal models with vector databases. This includes comprehensive coverage of component interactions, data flow patterns, and scalability considerations.

Section 4 provides detailed implementation specifics, including model configuration, database optimization, and system engineering decisions. This section is designed to enable reproduction and extension of our work.

Section 5 describes the user interface components, demonstrating how the underlying system capabilities are exposed for practical use cases.

Section 6 synthesizes our findings, discusses limitations, and positions our contributions within the broader context of multimodal retrieval research.

Section 7 summarizes key contributions and outlines directions for future research.

The appendices provide detailed technical documentation, setup procedures, and additional experimental results to support reproducibility and practical application of our work.

2 Background and Related Work

The proliferation of multimodal digital content has fundamentally challenged traditional information retrieval paradigms, necessitating novel approaches that can effectively bridge semantic gaps between visual and textual modalities. This section provides a comprehensive review of the theoretical foundations and technological advances that underpin our multimodal search engine implementation, positioning our contributions within the broader context of information retrieval, multimodal learning, and vector database research.

2.1 Information Retrieval Fundamentals

2.1.1 Traditional IR Models

Classical information retrieval systems have evolved through several paradigmatic shifts, each addressing fundamental limitations of their predecessors while introducing new computational and representational challenges. The Boolean Model, as one of the earliest formal approaches, established the foundation of exact-match retrieval through logical expressions composed of Boolean operators (AND, OR, NOT) [21]. While computationally tractable and conceptually transparent, this model's binary relevance assessment (documents are either entirely relevant or irrelevant) fundamentally limits its applicability to nuanced semantic matching scenarios.

The introduction of ranked retrieval marked a significant advancement, recognizing that relevance exists on a continuum rather than as a binary classification. The Vector Space Model (VSM) formalized this intuition by representing both documents and queries as real-valued vectors in $\mathbb{R}^{|V|}$, where $|V|$ denotes the vocabulary size [21, 29]. Document-query similarity is computed through geometric operations, typically cosine similarity:

$$\text{sim}(\mathbf{d}, \mathbf{q}) = \frac{\mathbf{d} \cdot \mathbf{q}}{\|\mathbf{d}\| \cdot \|\mathbf{q}\|} = \frac{\sum_{i=1}^{|V|} d_i q_i}{\sqrt{\sum_{i=1}^{|V|} d_i^2} \sqrt{\sum_{i=1}^{|V|} q_i^2}}, \quad (1)$$

where d and q are two documents/queries in the collection, and d_i and q_i is the importance of the i^{th} term in the document/query.

Term weighting schemes, particularly TF-IDF, provide the foundation for vector construction by balancing term frequency against inverse document frequency [14]:

$$w_{t,d} = (1 + \log_{10} \text{tf}_{t,d}) \log_{10} \frac{N}{\text{df}_t}, \quad (2)$$

where $\text{tf}_{t,d}$ is the term frequency in the document, N is the number of documents in the collection, and df_t is the inverse document frequency (idf). This approach effectively captures

the intuition that terms frequent in a document but rare across the collection carry greater discriminative power. Probabilistic models introduced a more theoretically grounded framework through the Probability Ranking Principle, which posits optimal retrieval performance when documents are ranked by their probability of relevance [21, 27]. The evolution toward Okapi BM25 incorporated term frequency normalization and document length considerations, establishing one of the most enduring ranking functions in information retrieval [28]:

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{\text{tf}(t, d) \cdot (k_1 + 1)}{\text{tf}(t, d) + k_1 \cdot (1 - b + b \cdot \frac{|d|}{\text{avgtl}})} \quad (3)$$

where k_1 and b are tuning parameters that control term frequency saturation and document length normalization, respectively. Language Models represent another significant theoretical advancement, treating documents as generative sources for query terms [25]. Smoothing techniques, such as Jelinek-Mercer and Dirichlet smoothing, address data sparsity by interpolating document-specific term probabilities with collection-wide statistics, enabling robust estimation even for unseen terms [34].

2.1.2 Neural Information Retrieval

The emergence of neural approaches in information retrieval represents a paradigmatic shift from symbolic to distributed representations, fundamentally altering how semantic similarity is conceptualized and computed. Traditional sparse retrieval methods, while effective for lexical matching, fail to capture semantic relationships between terms that do not share explicit surface forms. Neural IR addresses this limitation through learned dense representations that encode semantic meaning in continuous vector spaces.

The transition from sparse to dense retrieval can be formalized as a mapping from discrete, high-dimensional sparse vectors to continuous, lower-dimensional dense representations. Where traditional methods represent documents as $\mathbf{d} \in \{0, 1\}^{|V|}$ or $\mathbf{d} \in \mathbb{R}_+^{|V|}$ with predominantly zero entries, neural approaches learn mappings $f : \mathcal{D} \rightarrow \mathbb{R}^d$ where $d \ll |V|$ and vector components are continuous-valued.

Early neural IR approaches focused on learning improved term representations through word embeddings [24, 22], enabling semantic similarity computation between individual terms. However, the compositional challenge of aggregating word-level semantics into document-level representations remained. Approaches such as averaged word embeddings, though computationally efficient, fail to capture complex syntactic and semantic interactions within longer texts.

The development of contextualized embeddings through transformer architectures marked a fundamental breakthrough [32, 7]. Unlike static word embeddings, contextualized representations capture token semantics as functions of their surrounding context, enabling nuanced understanding of polysemy, syntactic relationships, and discourse-level phenomena. The attention mechanism, central to transformer architectures [2], enables selective focus on relevant portions of input sequences [32]:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (4)$$

where Q , K , and V represent query, key, and value matrices, respectively, and d_k denotes the dimensionality of key vectors.

2.1.3 From Sparse to Dense Retrieval

The architectural evolution from sparse to dense retrieval necessitates fundamental reconsiderations of indexing, similarity computation, and system design. Dense retrieval systems typically employ bi-encoder architectures where queries and documents are encoded independently through shared or specialized neural networks [15]:

$$\mathbf{e}_q = \text{Encoder}_q(q), \quad \mathbf{e}_d = \text{Encoder}_d(d) \quad (5)$$

Similarity is computed through inner product or cosine similarity in the learned embedding space:

$$\text{sim}(q, d) = \mathbf{e}_q^T \mathbf{e}_d \quad (6)$$

This architectural choice enables efficient pre-computation and indexing of document embeddings, supporting sub-linear search complexity through approximate nearest neighbor techniques. However, the independence assumption inherent in bi-encoders limits their ability to model fine-grained query-document interactions compared to cross-encoder architectures that process concatenated query-document pairs [3].

The training of dense retrieval systems typically employs contrastive learning objectives that maximize similarity between relevant query-document pairs while minimizing similarity for irrelevant pairs [15, 23].

The integration of dense retrieval with traditional sparse methods has emerged as a promising research direction, leveraging the complementary strengths of both approaches. Hybrid systems often employ sparse retrieval for initial candidate selection followed by dense re-ranking, or learn to combine sparse and dense similarity scores through learned fusion mechanisms [16, 30].

2.2 Multimodal Learning and Representation

2.2.1 Vision-Language Models

The convergence of computer vision and natural language processing has given rise to sophisticated vision-language models capable of learning joint representations across visual and textual modalities. These models address the fundamental challenge of semantic alignment between disparate data types, enabling cross-modal understanding and generation

tasks that were previously intractable.

Early approaches to vision-language integration relied on relatively simple fusion mechanisms, such as feature concatenation or element-wise operations between independently processed visual and textual features. However, these methods failed to capture the complex, nuanced relationships between visual concepts and their linguistic descriptions, limiting their effectiveness for sophisticated multimodal understanding tasks.

The introduction of attention-based fusion mechanisms marked a significant advancement, enabling models to selectively focus on relevant portions of both visual and textual inputs. Cross-modal attention allows the model to attend to specific image regions when processing textual tokens, and conversely, to focus on relevant textual concepts when processing visual features [19, 31]:

$$\text{CrossAttention}(V, T) = \text{softmax} \left(\frac{VW_q(TW_k)^T}{\sqrt{d}} \right) VW_v \quad (7)$$

where V and T represent visual and textual features, respectively, and W_q , W_k , W_v are learned projection matrices.

The BLIP family of models, particularly BLIP-2 as implemented in our system, represents a significant advancement in vision-language modeling through its innovative approach to bridging frozen pre-trained encoders [17]. The architecture introduces a trainable Q-Former module that serves as an information bottleneck between a frozen image encoder and a frozen language model [18]:

$$\mathbf{h}_{visual} = \text{Q-Former}(\text{ImageEncoder}(\mathbf{I}), \mathbf{Q}) \quad (8)$$

where $\mathbf{I}$ represents the input image, $\mathbf{Q}$ denotes learnable query embeddings, and $\mathbf{h}_{visual}$ is the resulting visual representation.

The Q-Former employs a sophisticated training regime encompassing three primary objectives:

Image-Text Contrastive Loss (ITC): Aligns visual and textual representations by maximizing similarity between matching pairs while minimizing similarity for non-matching pairs [17, 18, 26].

Image-to-Text Generation Loss (ITG): Encourages the extraction of comprehensive visual information necessary for text generation through causal language modeling objectives applied to textual descriptions [17, 18].

Image-Text Matching Loss (ITM): Trains binary classification to distinguish between aligned and misaligned image-text pairs, incorporating hard negative mining to improve discriminative capabilities [17, 18].

2.2.2 Cross-Modal Embedding Spaces

The construction of effective cross-modal embedding spaces requires careful consideration of geometric properties, semantic preservation, and computational efficiency. The fundamental challenge lies in learning mappings that preserve semantic relationships both within and across modalities while maintaining metric properties necessary for efficient similarity search [12].

Theoretical frameworks for cross-modal embedding spaces often draw upon metric learning principles, where the embedding function $f : \mathcal{X} \rightarrow \mathbb{R}^d$ preserves distance relationships:

$$d_{\mathcal{X}}(x_1, x_2) \approx d_{\mathbb{R}^d}(f(x_1), f(x_2)) \quad (9)$$

where $d_{\mathcal{X}}$ represents semantic distance in the original space and $d_{\mathbb{R}^d}$ denotes Euclidean or cosine distance in the embedding space.

The dimensionality of embedding spaces presents fundamental trade-offs between expressiveness and computational efficiency. Higher-dimensional spaces provide greater representational capacity but incur increased storage costs and computational complexity for similarity search. Our implementation employs 256-dimensional embeddings [20, 8, 13] as generated by BLIP-2, representing a carefully optimized balance for practical applications.

Normalization strategies significantly impact the geometric properties of embedding spaces. L2 normalization, as implemented in our system, constrains embeddings to the unit hypersphere, enabling cosine similarity computation through efficient inner products [4, 26]:

$$\text{cosine}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \cdot \|\mathbf{v}\|} = \mathbf{u}_{\text{norm}} \cdot \mathbf{v}_{\text{norm}} \quad (10)$$

This normalization also mitigates issues related to embedding magnitude variations across different input types or model outputs.

2.2.3 Contrastive Learning Approaches

Contrastive learning has emerged as the dominant paradigm for training cross-modal embedding models, providing a principled approach to learning semantic similarity functions without requiring dense annotation of similarity relationships [4]. The core principle involves maximizing agreement between positive pairs while minimizing agreement between negative pairs, effectively shaping the embedding space to reflect semantic relationships.

The InfoNCE loss, widely adopted in contrastive learning, provides a probabilistic framework for this optimization [23].

The effectiveness of contrastive learning critically depends on the quality and diversity of negative examples. Hard negative mining, where challenging negative examples are preferentially selected, has proven particularly effective for improving model discriminative capa-

bilities [18]. Our system’s training incorporates such strategies through the ITM objective, which explicitly constructs hard negatives for binary classification tasks.

Batch construction strategies significantly impact contrastive learning effectiveness. Large batch sizes increase the number of negative examples per positive pair, improving gradient estimation quality [4]. However, computational constraints often necessitate techniques such as gradient accumulation or memory banks to achieve effective batch sizes [10].

2.3 Vector Databases and Similarity Search

2.3.1 High-Dimensional Indexing

The efficient indexing and retrieval of high-dimensional vectors presents fundamental algorithmic and systems challenges that become increasingly pronounced as dimensionality increases. Traditional indexing structures, such as B-trees and their spatial extensions (R-trees, KD-trees), suffer from the curse of dimensionality, exhibiting degraded performance when applied to high-dimensional spaces characteristic of neural embeddings.

The curse of dimensionality manifests in several ways that directly impact retrieval system performance. As dimensionality increases, the volume of hyperspheres grows exponentially relative to hypercubes, causing most points to concentrate near the boundaries of bounded regions. This phenomenon leads to increased distance variance and reduced discriminative power of distance-based similarity measures [1]. For uniform random distributions in d -dimensional unit hypercubes, the expected distance between random points approaches:

$$\mathbb{E}[\|x - y\|_2] \approx \frac{\sqrt{d}}{3} \quad (11)$$

The relative variance of pairwise distances decreases as $O(1/d)$, making distance-based discrimination increasingly challenging in very high dimensions.

Modern indexing approaches address these challenges through several key strategies:

Locality-Sensitive Hashing (LSH) [6]: LSH methods construct hash functions that preserve locality, ensuring that similar vectors have a high probability of collision.

Hierarchical Navigable Small World (HNSW) [20]: HNSW constructs multi-layer graphs where higher layers contain fewer nodes with long-range connections, enabling efficient approximate nearest neighbor search through greedy navigation.

Product Quantization (PQ) [12]: PQ compresses vectors through learned quantization, decomposing d -dimensional vectors into m subvectors of dimension d/m [12]. Each subvector is quantized using a learned codebook, enabling compact representation and efficient distance computation through lookup tables. Our implementation employs HNSW indexing through PostgreSQL’s pgvector extension, which provides optimized implementations of both exact and approximate similarity search [20].

2.3.2 Approximate Nearest Neighbor Search

Approximate Nearest Neighbor (ANN) search algorithms provide essential infrastructure for large-scale similarity search by trading modest accuracy reductions for significant computational speedups [9]. The theoretical foundation of ANN algorithms rests on the observation that exact nearest neighbor search in high dimensions often provides limited practical benefit compared to high-quality approximate solutions.

The performance characteristics of ANN algorithms depend critically on intrinsic dimensionality, data distribution, and query patterns. Datasets with low intrinsic dimensionality, despite high nominal dimensionality, often exhibit superior ANN performance due to improved separability in the underlying manifold structure.

Graph-based ANN: Graph-based approaches, exemplified by HNSW, construct connectivity graphs where edges connect similar vectors [20]. Search proceeds through greedy navigation [20] from random entry points, maintaining candidate sets and updating best-found neighbors. The multi-layer structure of HNSW enables efficient search by providing coarse-to-fine navigation paths. The construction algorithm for HNSW inserts vectors sequentially, determining layer assignment through geometric distributions.

Tree-based ANN: Despite their limitations in high dimensions, tree-based approaches remain relevant for specific scenarios. Random projection trees partition space through random hyperplanes, creating binary trees where leaves contain small point sets [11]. Multiple trees provide improved coverage and recall through ensemble effects.

Quantization-based ANN: Methods such as Product Quantization and Optimized Product Quantization compress vectors while preserving distance relationships [12]. The quantization error introduces approximation, but the compressed representation enables memory-efficient storage and accelerated distance computation.

Our system's integration with pgvector leverages optimized HNSW implementations that provide excellent performance for 256-dimensional embeddings. The configuration parameters, including the number of connections per node and the size of the dynamic candidate list, are tuned based on the specific characteristics of BLIP-2 embeddings and expected query patterns.

2.3.3 PostgreSQL and pgvector Extension

PostgreSQL ¹ with the pgvector extension represents a compelling integration of traditional relational database capabilities with modern vector similarity search functionality. This combination addresses a critical gap in the vector database ecosystem by providing ACID transactions, mature query optimization, and extensive ecosystem support alongside efficient similarity search.

The pgvector extension introduces a dedicated vector data type and associated operators for similarity computation:

¹<https://github.com/pgvector/pgvector>

```
1 -- Vector type declaration
2 CREATE TABLE embeddings (
3     id SERIAL PRIMARY KEY,
4     vector VECTOR(256),
5     metadata JSONB
6 );
7
8 -- Similarity operators
9 SELECT * FROM embeddings
10 ORDER BY vector <-> '[0.1,0.2,...]':vector
11 LIMIT 10;
```

The extension supports multiple distance metrics $\langle - \rangle$:

- Euclidean distance (L_2);
- Cosine distance;
- Inner product;
- L_1 distance;
- Hamming distance;
- Jaccard distance.

3 System Architecture

The multimodal search engine presented in this work represents a sophisticated integration of state-of-the-art vision-language models with a scalable vector database infrastructure. This section provides a comprehensive analysis of the system's architectural design, examining both the high-level component interactions and the detailed implementation strategies that enable efficient multimodal retrieval at scale. The architecture addresses fundamental challenges in cross-modal information retrieval while maintaining the modularity and extensibility necessary for research and production deployment scenarios.

3.1 Overall System Design

3.1.1 Component Overview

The system architecture follows a layered design pattern that separates concerns across three primary abstraction levels: presentation, application logic, and data persistence. This architectural choice facilitates independent optimization of each layer while maintaining clear interfaces that enable component substitution and extension.

At the presentation layer, the system provides dual interface modalities: a command-line interface (CLI) designed for batch processing, automated workflows, and integration with external systems, and a web-based interface implemented using Gradio that enables interactive exploration and real-time query processing. This dual-interface approach addresses the diverse usage patterns encountered in both research and practical deployment scenarios.

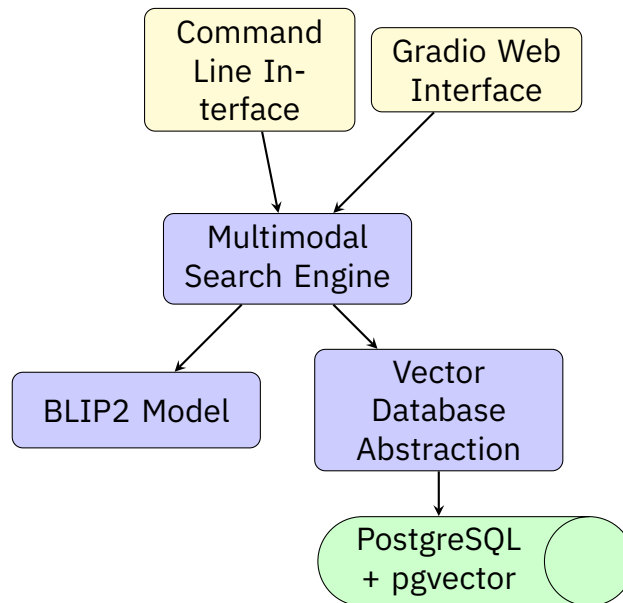


Figure 1: High-level system architecture showing the layered design with clear separation between presentation, application logic, and data persistence layers.

The application logic layer centers around the `MultimodalSearchEngine` class, which orchestrates the complex interactions between embedding generation, similarity computation, and result aggregation. This component implements a facade pattern that abstracts the underlying complexity of multimodal processing while providing a unified interface for diverse query types. The engine coordinates between the BLIP2 [17] model for embedding generation and the vector database abstraction for similarity search, implementing sophisticated caching and optimization strategies to minimize computational overhead.

The data persistence layer employs a modular design through the `VDBConfig` and vector database abstraction, enabling seamless integration of different vector database backends. The current implementation focuses on PostgreSQL with pgvector extension, chosen for its mature ecosystem, ACID compliance, and excellent performance characteristics for moderate-dimensional embeddings. However, the abstraction layer facilitates future integration of specialized vector databases such as Milvus, Pinecone, or Weaviate as requirements evolve.

3.1.2 Data Flow Architecture

The system's data flow architecture (Fig. 2) implements a carefully orchestrated pipeline that transforms raw multimodal inputs into semantically meaningful search results. The architecture addresses several critical challenges: maintaining consistency between different modalities, optimizing computational resource utilization, and ensuring scalable query processing under varying load conditions.

The data flow pipeline begins with intelligent query type detection that automatically determines the optimal processing path based on input characteristics. For image queries, the system invokes the image encoding pipeline, which applies preprocessing transformations, forwards the processed image through the BLIP2 visual encoder, and generates normalized

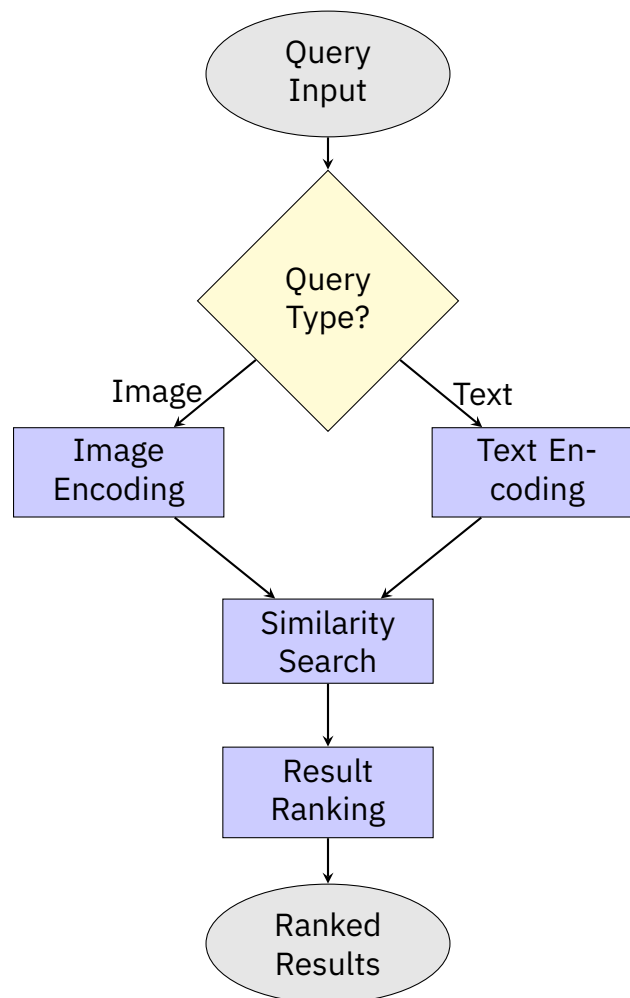


Figure 2: Data flow pipeline showing the processing stages from query input to ranked results, with parallel processing paths for different modalities.

embedding vectors. Text queries follow a parallel path through the text encoding pipeline, utilizing the BLIP2 language components to produce semantically equivalent text embeddings.

A critical architectural decision involves the handling of multimodal queries that combine both image and text components. The system implements a fusion mechanism that processes each modality independently. This approach maintains the semantic properties of individual modalities while enabling flexible query composition.

The similarity search component implements optimized vector operations that leverage the geometric properties of L2-normalized embeddings. By constraining embeddings to the unit hypersphere, cosine similarity computation reduces to efficient inner product operations, enabling significant computational savings during large-scale similarity search.

Result ranking and aggregation represent sophisticated processes that extend beyond simple similarity scoring. The system implements a hierarchical scoring mechanism that accounts for product-variant relationships, aggregates similarity scores across multiple embeddings per product, and applies relevance-based re-ranking strategies. This approach addresses the practical requirement of returning coherent product-level results rather than individual embedding matches.

3.2 Multimodal Embedding Generation

3.2.1 BLIP2 Model Architecture

The system leverages BLIP2 (Bootstrapped Language-Image Pre-training 2) [17] as its foundational multimodal model, representing a significant advancement in vision-language understanding through its innovative architectural design. BLIP2 addresses fundamental limitations of previous approaches by introducing a trainable Q-Former module that serves as an information bottleneck between frozen pre-trained vision and language encoders.

The architectural innovation of BLIP2 lies in its ability to leverage powerful pre-trained models without requiring expensive end-to-end fine-tuning. The system employs frozen components from established models: a ViT (Vision Transformer) for visual encoding and a BERT-based model for text processing. The Q-Former module, implemented as a lightweight transformer, learns to extract and align relevant information from both modalities.

Our implementation utilizes a modified BLIP2 architecture, as evidenced in the `Blip2ITM` class, which extends the base BLIP2 model with specialized methods for independent modality encoding:

```
1 def forward_image(self, image):
2     with self.maybe_autocast():
3         image_embeds = self.ln_vision(self.visual_encoder(image)).float()
4         image_atts = torch.ones(image_embeds.size()[:-1],
5                                 dtype=torch.long).to(image.device)
6
7         query_tokens = self.query_tokens.expand(image_embeds.shape[0], -1, -1)
8         query_output = self.Qformer.bert(
9             query_embeds=query_tokens,
```

Multimodal Search Engine

```

10         encoder_hidden_states=image_embeds,
11         encoder_attention_mask=image_atts,
12         return_dict=True,
13     )
14
15     image_feats = F.normalize(
16         self.vision_proj(query_output.last_hidden_state), dim=-1
17     )
18     return image_feats

```

This architectural modification enables independent encoding of visual and textual inputs, facilitating efficient batch processing and enabling the bi-encoder paradigm essential for large-scale retrieval applications. The normalized output ensures that similarity computation reduces to efficient inner product operations.

The Q-Former component implements learnable query tokens that interact with visual features through cross-attention mechanisms. These queries learn to extract the most relevant visual information for cross-modal alignment, effectively serving as a learned information bottleneck that distills complex visual scenes into compact, semantically meaningful representations.

3.2.2 Image Encoding Pipeline

The image encoding pipeline implements a sophisticated preprocessing and encoding strategy designed to maximize semantic content extraction while maintaining computational efficiency. The pipeline addresses several critical challenges: input normalization across diverse image formats, resolution optimization for the underlying ViT architecture, and embedding post-processing for optimal similarity search performance.

The preprocessing stage employs the LAVIS library's `blip_image_eval` processor, which implements a carefully designed transformation sequence:

$$\mathbf{I}_{processed} = \text{VisualProcessor}(\mathbf{I}_{raw}) \quad (12)$$

The conversion step ensures consistent RGB format, addressing variations in input color spaces and channel orderings. The resize operation transforms input images to either 224×224 pixels (for the pre-trained variant) or 364×364 pixels (for the COCO-fine-tuned variant). The processed image $\mathbf{I}_{processed}$ is then projected into the latent space through the BLIP2 model.

3.2.3 Text Encoding Pipeline

The text encoding pipeline implements a parallel processing strategy that transforms textual queries into semantically equivalent vector representations. The pipeline addresses unique challenges associated with text processing: tokenization, sequence length, and latent space projections.

Sequence length management implements truncation strategies that preserve the most semantically relevant portions of input text. The system applies a maximum sequence length of `max_txt_len`, with truncation applied to the end of sequences to preserve initial con-

textual information:

```
1 text = self.tokenizer(  
2     caption,  
3     truncation=True,  
4     max_length=self.max_txt_len,  
5     return_tensors="pt",  
6 ).to(next(self.parameters()).device)
```

The encoding process forwards tokenized text through the Q-Former's BERT component, which implements bidirectional attention mechanisms that capture complex semantic relationships within the input text. The system extracts the [CLS] token representation, which serves as a learned aggregate representation of the entire sequence.

Projection through the text projection layer maps the hidden states to the shared multi-modal embedding space, ensuring semantic alignment with visual embeddings. The BLIP2 projection layer implements a learned linear transformation that optimizes cross-modal similarity while preserving intra-modal semantic relationships.

3.3 Vector Database Integration

3.3.1 PostgreSQL with pgvector Setup

The integration of PostgreSQL with the pgvector extension represents a strategic architectural choice that combines the maturity and reliability of traditional relational databases [5] with modern vector similarity search capabilities. This approach addresses a critical gap in the vector database ecosystem by providing ACID transactions, sophisticated query optimization, and extensive ecosystem support alongside efficient high-dimensional similarity search [9, 33].

The pgvector extension introduces native vector data types and operators that enable seamless integration of similarity search with traditional SQL operations. The extension supports multiple similarity metrics through specialized operators: L2 distance, inner product, cosine distance, L1 distance, Hamming distance, and Jaccard distance.

Our implementation primarily employs cosine similarity due to its invariance to embedding magnitude and superior empirical performance with L2-normalized embeddings. The mathematical properties of cosine similarity align naturally with the geometric constraints imposed by our embedding normalization strategy.

3.3.2 Database Schema Design

The database schema implements a carefully designed relational model that balances normalization principles with query performance requirements. The schema addresses the complex relationships between products, variants, and multimodal embeddings while maintaining referential integrity and enabling efficient similarity search operations.

The core schema consists of four primary tables that capture the hierarchical relationship

between products and their variants:

```

1 CREATE TABLE products (
2     id TEXT PRIMARY KEY,
3     name TEXT NOT NULL
4 );
5
6 CREATE TABLE product_variant (
7     id_product TEXT,
8     id_variant TEXT,
9     name TEXT,
10    PRIMARY KEY (id_product, id_variant),
11    CONSTRAINT fk_product FOREIGN KEY (id_product)
12        REFERENCES product(id) ON DELETE CASCADE
13 );
14
15 CREATE TABLE image_embedding (
16     id SERIAL PRIMARY KEY,
17     id_product TEXT,
18     id_variant TEXT,
19     image_name TEXT,
20     embeddings VECTOR(256),
21    CONSTRAINT fk_image_variant FOREIGN KEY (id_product, id_variant)
22        REFERENCES product_variant(id_product, id_variant) ON DELETE CASCADE
23 );
24
25 CREATE TABLE text_embedding (
26     id SERIAL PRIMARY KEY,
27     id_product TEXT,
28     id_variant TEXT,
29     caption TEXT,
30     embeddings VECTOR(256),
31    CONSTRAINT fk_text_variant FOREIGN KEY (id_product, id_variant)
32        REFERENCES product_variant(id_product, id_variant) ON DELETE CASCADE
33 );

```

3.4 Search Engine Components

3.4.1 MultimodalSearchEngine Class

The `MultimodalSearchEngine` class serves as the central orchestrator for all multimodal retrieval operations, implementing a sophisticated facade pattern that abstracts the complexity of cross-modal similarity search while providing a unified interface for diverse query modalities. The class design emphasizes modularity, extensibility, and performance optimization through careful resource management and algorithmic choices.

The class initialization process addresses several critical concerns:

```

1 def __init__(self, pretrained_model: MultimodalPretrainedModel,
2             vdb_config: VDBConfig, device: torch.device):
3     # Device configuration and optimization
4     self.device = device
5
6     # Model initialization with optimal configuration
7     self.vis_processor = load_processor("blip_image_eval",
8         cfg={"image_size": 224 if pretrained_model ==
9             MultimodalPretrainedModel.PRETRAIN else 364})
10
11     # Model loading with device placement and precision optimization
12     self.blip2 = Blip2ITM.from_pretrained(pretrained_model.value)\

```

Multimodal Search Engine

```

13         .to(device).float()
14     self.blip2.eval()
15
16     # Database abstraction initialization
17     match vdb_config.type:
18         case VDBType.PGVECTOR.value:
19             self.db = PGVectorDB(vdb_config)
20         case _:
21             raise ValueError(f"Unsupported vector database type")

```

This initialization strategy ensures optimal configuration for the target deployment environment while maintaining flexibility for different hardware configurations and database backends.

Resource Management: The class implements sophisticated resource management strategies that address GPU memory limitations, database connection lifecycle, and embedding caching:

- **GPU Memory Optimization:** Inference operations employ `torch.no_grad()` contexts to disable gradient computation, reducing memory usage;
- **Model State Management:** The model is explicitly set to evaluation mode, ensuring consistent inference behavior and optimizing computational efficiency;
- **Database Connection Lifecycle:** Connection management is delegated to the database abstraction layer.

3.4.2 Product and Variant Management

The product and variant management system addresses the complex hierarchical relationships typical of real-world product catalogs, where individual products may have multiple variants distinguished by attributes such as color, size, style, or configuration. This system enables coherent product-level search results while maintaining the granular embedding storage necessary for accurate similarity computation.

The hierarchical data model implements a two-level structure:

$$\text{Product} \rightarrow \{\text{Variant}_1, \text{Variant}_2, \dots, \text{Variant}_n\} \quad (13)$$

Each variant maintains independent embeddings for both visual and textual modalities, enabling fine-grained similarity computation while supporting aggregation to product-level results.

The `add_product` method implements atomic product insertion with comprehensive validation:

```

1 def add_product(self, id: str, id_variant: str, name: str,
2                 image: str, captions: List[str],
3                 image_embed: torch.Tensor,
4                 text_embeds: List[torch.Tensor]) -> None:
5
6     # Atomic product insertion with existence checking
7     if not self.db.product_exists(id):

```

Multimodal Search Engine

```

8         self.db.insert_product(id, name)
9
10    if not self.db.product_variant_exists(id, id_variant):
11        self.db.insert_product_variant(id, id_variant, f"{id}_{id_variant}")
12
13    # Embedding validation and normalization
14    if image_embed.shape[1] != text_embeds[0].shape[1]:
15        image_embed = torch.transpose(image_embed, 1, 2)
16
17    # Multi-modal embedding insertion
18    self.db.insert_image_embedding(id, id_variant, image,
19                                  image_embed.squeeze(0).mean(dim=1))
20
21    for caption, text_embed in zip(captions, text_embeds):
22        self.db.insert_text_embedding(id, id_variant, caption,
23                                     text_embed.squeeze(0).mean(dim=1).view(-1))

```

This approach ensures data consistency through existence checking, handles dimensional mismatches between modalities through appropriate tensor operations, and supports multiple text embeddings per variant to capture diverse textual descriptions.

Variant Relationship Management: The system maintains explicit relationships between products and variants through foreign key constraints, enabling efficient navigation of the product hierarchy during search operations. This approach supports complex queries that may require product-level aggregation of variant-level similarity scores.

3.4.3 Similarity Computation

The similarity computation component implements optimized algorithms for cross-modal similarity assessment, addressing the geometric properties of embedding spaces while providing efficient computation for large-scale search operations. The implementation leverages the mathematical properties of L2-normalized embeddings to enable efficient similarity computation through inner product operations.

The core similarity computation employs cosine similarity, which provides several advantages for multimodal retrieval:

$$\text{sim}(\mathbf{e}_1, \mathbf{e}_2) = \frac{\mathbf{e}_1 \cdot \mathbf{e}_2}{\|\mathbf{e}_1\| \cdot \|\mathbf{e}_2\|} = \mathbf{e}_1 \cdot \mathbf{e}_2 \quad (14)$$

The final equality holds due to L2 normalization, reducing computational complexity from $O(3d)$ to $O(d)$ operations.

The `get_cosine_similarity` method handles dimensional mismatches that may occur between different modalities:

```

1 def get_cosine_similarity(self, embed_1: torch.Tensor,
2                           embed_2: torch.Tensor) -> float:
3     # Handle dimensional mismatches through transposition
4     if embed_1.shape[1] != embed_2.shape[1]:
5         embed_1 = torch.transpose(embed_1, 1, 2)
6
7     # Efficient cosine similarity computation
8     cosine_sim = F.cosine_similarity(embed_1, embed_2, dim=1)
9

```

Multimodal Search Engine

```

10     # Maximum similarity across multiple embeddings
11     max_sim = cosine_sim.max().item()
12
13     return max_sim

```

3.4.4 Result Ranking and Aggregation

The result ranking and aggregation system addresses the complex challenge of transforming individual embedding similarities into coherent, ranked product results. This system must handle hierarchical product-variant relationships, multi-modal query fusion, and relevance-based ranking while maintaining computational efficiency for real-time query processing.

The search method implements a sophisticated multi-stage process:

```

1  def search(self, image_embed: torch.Tensor = None,
2              text_embed: torch.Tensor = None,
3              alpha: float = 0.5, top_k: int = 10) -> List[ProductVariant]:
4
5      # Input validation
6      if image_embed is None and text_embed is None:
7          raise ValueError("Must provide at least one embedding")
8
9      # Initialize result aggregation structure
10     search_results = SearchResults()
11
12     # Multi-modal search processing
13     if image_embed is not None:
14         image_embed = image_embed.mean(dim=1).squeeze(0)
15         products_variants = self.db.search_product_variants_by_image(image_embed)
16
17         # Result aggregation and product hierarchy management
18         for id, id_variant, name, image_name, similarity in products_variants:
19             if not search_results.has_product(id):
20                 product = Product(id=id, name=name)
21                 search_results.add_product(product)
22
23                 search_results.get_product(id).add_image(id_variant, image_name, similarity)
24
25     # Text search processing with similar aggregation logic
26     if text_embed is not None:
27         text_embed = text_embed.view(-1)
28         products_variants = self.db.search_product_variants_by_caption(text_embed)
29
30         # Symmetric aggregation for text results
31         for id, id_variant, name, caption, similarity in products_variants:
32             if not search_results.has_product(id):
33                 product = Product(id=id, name=name)
34                 search_results.add_product(product)
35
36                 search_results.get_product(id).add_text(id_variant, caption, similarity)
37
38     # Final ranking and top-k selection
39     return search_results.get_topk_products(alpha, top_k)

```

Multi-Modal Fusion Strategy: The system implements a sophisticated fusion mechanism that combines similarities from different modalities through weighted linear combination. The alpha parameter controls the relative importance of visual versus textual similarity:

$$\text{score}_{combined} = \sqrt{\text{score}_{image}^{\alpha} \cdot \text{score}_{best_caption}^{(1-\alpha)}} \quad (15)$$

where score_{image} and $\text{score}_{best_caption}$ are the similarity scores for images and captions of queries and documents, whereas $\alpha \in [0, 1]$ represents a user-configurable parameter that controls the relative importance of visual versus textual components. This approach enables users to emphasize specific modalities based on query characteristics or application requirements.

4 Implementation Details

This section provides a comprehensive examination of the implementation methodology, architectural decisions, and optimization strategies employed in developing the multimodal search engine. The implementation addresses fundamental challenges in scaling neural information retrieval systems while maintaining research reproducibility and production viability. Our approach emphasizes modularity, performance optimization, and robust error handling throughout the system stack.

4.1 Development Environment and Dependencies

4.1.1 Software Stack

The implementation leverages a carefully curated software stack that balances cutting-edge research capabilities with production stability. The core framework is built upon PyTorch 2.0+, chosen for its dynamic computation graph, extensive ecosystem support, and superior performance characteristics for transformer-based architectures. PyTorch's automatic differentiation engine, while not directly utilized during inference, provides the foundation for potential fine-tuning scenarios and ensures compatibility with the broader deep learning research ecosystem.

The LAVIS (Language-Vision Intelligence) library serves as the foundation for multimodal model integration, providing optimized implementations of state-of-the-art vision-language models including BLIP2 [17]. LAVIS offers several critical advantages over direct model implementations:

- **Optimized Preprocessing Pipelines:** Pre-configured image and text processors that implement the exact transformations used during model pre-training, ensuring distribution alignment between inference and training data;
- **Model Versioning and Checkpoints:** Systematic management of pre-trained model weights;
- **Memory-Efficient Implementations:** Optimized model architectures.

The database integration employs PostgreSQL 17, representing the latest stable release with enhanced performance characteristics and improved JSON processing capabilities.

Multimodal Search Engine

The pgvector extension (version 0.8.0+) provides native vector similarity search capabilities with ACID transaction support, enabling complex queries that combine vector similarity with traditional relational operations. This integration eliminates the architectural complexity of maintaining separate vector and metadata databases, reducing operational overhead and improving query consistency.

Additional critical dependencies include:

- **NumPy 1.24+:** Optimized linear algebra operations with BLAS/LAPACK integration for efficient tensor manipulations outside PyTorch contexts;
- **Pillow 10.0+:** Robust image processing with support for diverse formats and color spaces, essential for handling heterogeneous input data;
- **psycopg 3.2+:** High-performance PostgreSQL adapter with native C extensions for optimal database connectivity;
- **Click 8.1+:** Command-line interface framework enabling sophisticated argument parsing and validation;
- **Gradio 5.23+:** Web interface framework optimized for machine learning applications with support for real-time updates and asynchronous processing.

4.1.2 Dependency Management with Conda

The project employs Conda for dependency management, providing reproducible environments across diverse hardware configurations and operating systems. The environment specification addresses the complex dependency relationships inherent in deep learning projects, particularly the intricate interactions between CUDA versions, PyTorch builds, and system libraries.

The `environment.yaml` specification implements a hierarchical dependency resolution strategy:

```
1 name: multimodal-search
2 channels:
3   - pytorch
4   - conda-forge
5   - defaults
6 dependencies:
7   - python=3.10
8   - requests=2.32.3
9   - click=8.1.7
10  - pytorch=2.1.0
11  - torchvision=0.16.0
12  - cpuonly
13  - numpy=1.24.3
14  - matplotlib=3.7.2
15  - pillow=10.0.1
16  - psycopg=3.2.6
17  - pgvector-python=0.4.0
```

Multimodal Search Engine

```
18 - omegaconf=2.3.0
19 - pypdf=5.4.0
20 - python-dotenv=1.1.0
21 - pydantic=2.10.6
22 - pip:
23   - log-candy==0.1.2
24   - opencv-contrib-python==4.10.0.84
25   - google-generativeai==1.8.0
26   - python-dotenv==1.1.0
27   - gradio==5.23.3
28   - git+https://github.com/salesforce/LAVIS.git@refs/pull/368/merge
```

This configuration strategy provides several advantages:

Channel Prioritization: The pytorch channel receives highest priority for core deep learning components, ensuring compatibility between PyTorch, torchvision, and CUDA toolkits.

Version Constraints: Minimum version specifications ensure compatibility with required features while allowing automatic updates for bug fixes and security patches. Critical dependencies employ exact version pinning in production deployments.

Mixed Package Management: The hybrid approach combines Conda's robust dependency resolution for system-level packages with pip installation for specialized packages unavailable through Conda channels.

4.2 Data Processing Pipeline

4.2.1 Dataset Preparation

The data processing pipeline implements a sophisticated multi-stage approach that addresses the heterogeneous nature of multimodal datasets while maintaining scalability and reproducibility. The pipeline supports multiple dataset formats, with specialized handling for research datasets (COCO) and domain-specific collections (Natuzzi furniture catalog).

4.2.2 Embedding Generation Process

The embedding generation process represents the most computationally intensive component of the system.

Batch Processing Optimization: The system implements dynamic batch size adaptation based on available GPU memory and input characteristics:

```
1 def compute_embeddings_coco(image_text_embedding_engine, embeddings_filename, limit):
2     # Containers for batch accumulation
3     all_image_embeddings = []
4     all_text_embeddings = []
5     all_captions_used = []
6
7     # Process dataset with progress monitoring
8     for img_pil, captions_list in tqdm(coco_subset, desc="Computing embeddings"):
```

Multimodal Search Engine

```

9      # Image embedding generation with memory optimization
10     image_embed = image_text_embedding_engine.encode_image(img_pil)
11     all_image_embeddings.append(image_embed.cpu()) # Move to CPU for storage
12
13     # Multi-caption text processing
14     selected_captions = captions_list[:num_captions_to_use] if len(captions_list) >=
        num_captions_to_use else captions_list
15
16     image_text_embeds = []
17     for caption in selected_captions:
18         text_embed = image_text_embedding_engine.encode_text(caption)
19         image_text_embeds.append(text_embed.cpu())
20
21     all_text_embeddings.append(image_text_embeds)
22     all_captions_used.append(selected_captions)

```

The CPU offloading strategy (`.cpu()`) prevents GPU memory accumulation during long-running batch processing operations, enabling processing of datasets larger than available GPU memory.

All inference operations execute within `torch.no_grad()` contexts, eliminating gradient computation overhead and reducing memory usage

4.2.3 Database Population

The database population process addresses the challenge of efficiently inserting large volumes of high-dimensional vectors while maintaining transactional consistency and optimal index performance. The implementation employs sophisticated strategies for batch insertion, index management, and error recovery.

Transactional Batch Insertion: The system implements atomic batch insertion with comprehensive error handling:

```

1  def populate_db(multimodal_search_engine: MultimodalSearchEngine, embeddings_filename: str) -> None:
2      """
3      Populates the database with the computed embeddings.
4
5      Args:
6          multimodal_search_engine (MultimodalSearchEngine): Model for generating embeddings.
7          embeddings_path (str): Path to the embeddings file.
8      """
9
10     # Load the embeddings
11     output = torch.load(f"embeddings/{embeddings_filename}")
12
13     # Reset the database
14     multimodal_search_engine.db.reset_db()
15
16     # For each product, add it to the database
17     for i in range(len(output['image_embeddings'])):
18
19         # Ensure the data in the .pt are valid, if not warn the user to recompute the embeddings
20         if 'ids' not in output or 'names' not in output or 'image_names' not in output or '
            image_embeddings' not in output or 'text_embeddings' not in output:
21             log_error("Invalid data in the embeddings file. Please recompute the embeddings.")
22             raise ValueError("Invalid data in the embeddings file. Please recompute the embeddings."
                )
23
24     # Get the data
25     id = output['ids'][i]
26     name = output['names'][i]

```

Multimodal Search Engine

```
27     image_name = output['image_names'][i]
28     captions = output['captions'][i]
29     image_embedding = output['image_embeddings'][i]
30     texts_embeddings = output['text_embeddings'][i]
31
32     # Get product variant
33     product_variant = Path(image_name).stem.split("_")[1]
34
35     # Add the product to the database
36     multimodal_search_engine.add_product(
37         id=id,
38         id_variant=product_variant,
39         name=name,
40         image=image_name,
41         captions=captions,
42         image_embed=image_embedding,
43         text_embeds=texts_embeddings,
44     )
```

4.3 Model Configuration and Optimization

4.3.1 BLIP2 Model Variants

The system supports multiple BLIP2 model variants, each optimized for different deployment scenarios and performance requirements. The variant selection impacts computational complexity, memory usage, and embedding quality, requiring careful consideration of application-specific trade-offs.

Pre-trained Variant (224×224): The base pre-trained variant employs 224×224 image resolution, representing the optimal balance between computational efficiency and semantic quality for general-purpose applications:

```
1 # Pre-trained variant configuration
2 self.vis_processor = load_processor("blip_image_eval",
3     cfg={"image_size": 224 if pretrained_model == MultimodalPretrainedModel.PRETRAIN else 364})
4
5 self.blip2 = Blip2ITM.from_pretrained(pretrained_model.value).to(device).float()
```

This variant provides:

- **Memory Efficiency:** Reduced memory footprint enabling deployment on consumer-grade GPUs;
- **Inference Speed:** Optimized for real-time query processing with sub-second response times;
- **General Applicability:** Broad semantic understanding across diverse visual domains.

COCO Fine-tuned Variant (364×364): The COCO-specific variant employs higher resolution inputs and domain-specific fine-tuning. The 364×364 resolution provides enhanced detail preservation at the cost of increased computational requirements, optimal for applications requiring fine-grained visual understanding.

Model Architecture Modifications: Our implementation extends the base BLIP2 architecture with specialized methods for independent modality encoding:

```

1 class Blip2ITM(Blip2ITMBase):
2     def forward_image(self, image):
3         with self.maybe_autocast():
4             image_embeds = self.ln_vision(self.visual_encoder(image)).float()
5             image_atts = torch.ones(image_embeds.size()[:-1], dtype=torch.long).to(image.device)
6             query_tokens = self.query_tokens.expand(image_embeds.shape[0], -1, -1)
7             query_output = self.Qformer.bert(
8                 query_embeds=query_tokens,
9                 encoder_hidden_states=image_embeds,
10                encoder_attention_mask=image_atts,
11                return_dict=True,
12            )
13            image_feats = F.normalize(
14                self.vision_proj(query_output.last_hidden_state), dim=-1
15            )
16            return image_feats
17
18     def forward_text(self, caption):
19         text = self.tokenizer(
20             caption,
21             truncation=True,
22             max_length=self.max_txt_len,
23             return_tensors="pt",
24         ).to(next(self.parameters()).device)
25         text_output = self.Qformer.bert(
26             text.input_ids,
27             attention_mask=text.attention_mask,
28             return_dict=True,
29         )
30         text_feat = F.normalize(
31             self.text_proj(text_output.last_hidden_state[:, 0, :]), dim=-1
32         )
33         return text_feat.unsqueeze(-1)

```

These modifications enable independent processing of visual and textual inputs, essential for the bi-encoder architecture required by large-scale retrieval systems.

4.4 Database Operations

4.4.1 Connection Management

The database connection management system addresses the challenges of maintaining robust, high-performance database connectivity in both development and production environments. The implementation employs sophisticated connection pooling, error recovery, and performance monitoring strategies.

Connection Pool Architecture: The system implements connection pooling through psycopg's built-in pool management:

```

1 class PGVectorDB:
2     def __init__(self, vdb_config: VDBConfig):
3         """
4         Initializes the PGVectorDB instance with the specified database type and configuration.
5
6         Args:
7             vdb_type (VDBType): The type of the vector database.
8             vdb_config (VDBConfig): The configuration for the vector database.
9         """
10        self._init_vdb(vdb_config)
11

```

Multimodal Search Engine

```

12  def _init_vdb(self, vdb_config: VDBConfig) -> None:
13      """
14      Initializes the vector database.
15
16      Args:
17          vdb_config (VDBConfig): Configuration for the vector database.
18      """
19
20      # Create the connection to the database
21      self.db = psycopg.connect(
22          dbname=vdb_config.db_name,
23          user=vdb_config.username,
24          password=vdb_config.password,
25          host=vdb_config.host,
26          port=vdb_config.port,
27          autocommit=True
28      )
29
30      # Init the vector database if needed
31      with self.db.cursor() as cur:
32
33          # Check if the products table exists
34          cur.execute(f"SELECT to_regclass('public.product')")
35          products_table_exists = cur.fetchone()[0]
36
37          # If the products is empty, reset the database
38          if not products_table_exists:
39              self.reset_db()

```

4.4.2 Vector Similarity Search

The vector similarity search implementation leverages pgvector's optimized algorithms while providing application-specific optimizations for multimodal retrieval scenarios.

Image Similarity Search: The system implements optimized image-based similarity queries:

```

1  def search_product_variants_by_image(self, image_embed: torch.Tensor) -> List[tuple]:
2      """
3      Searches for product variants based on the provided image embedding.
4
5      Args:
6          image_embedding (torch.Tensor): The image embedding to search for similar product
7              variants.
8
9      Returns:
10         List[tuple]: A list of tuples containing the product ID, variant ID, name, image name,
11             and similarity score.
12     """
13     with self.db.cursor() as cur:
14         cur.execute(f"""
15             SELECT p.id_product, p.id_variant, p.name, i.image_name, (1 - (i.embeddings
16                 <=> %s::vector)) AS similarity
17             FROM {DBTable.PRODUCTS_VARIANTS} p
18             JOIN {DBTable.IMAGE_EMBEDDINGS} i ON p.id_product = i.id_product AND p.
19                 id_variant = i.id_variant
20             ORDER BY similarity DESC
21             """, (image_embed.tolist(),))
22     return cur.fetchall()

```

Text Similarity Search: Parallel implementation for text-based queries:

```

1  def search_product_variants_by_caption(self, text_embed: torch.Tensor) -> List[tuple]:

```

Multimodal Search Engine

```

2      """
3      Searches for product variants based on the provided caption embedding.
4
5      Args:
6          text_embed (torch.Tensor): The caption embedding to search for similar product variants.
7
8      Returns:
9          List[tuple]: A list of tuples containing the product ID, variant ID, name, caption, and
10                      similarity score.
11      """
12      with self.db.cursor() as cur:
13          cur.execute(f"""
14              SELECT p.id_product, p.id_variant, p.name, t.caption, (1 - (t.embeddings <=>
15              %s::vector)) AS similarity
16              FROM {DBTable.PRODUCTS_VARIANTS} p
17              JOIN {DBTable.TEXT_EMBEDDINGS} t ON p.id_product = t.id_product AND p.
18                  id_variant = t.id_variant
19              ORDER BY similarity DESC
20              """, (text_embed.tolist(),))
21      return cur.fetchall()

```

Database Reset Operations: The system implements safe database reset operations for development and testing:

```

1  def reset_db(self) -> None:
2      """
3      Resets the database by dropping and recreating the tables.
4      """
5
6      # Log
7      log_info("Resetting the database...")
8
9      with self.db.cursor() as cur:
10         cur.execute('CREATE EXTENSION IF NOT EXISTS vector')
11         register_vector(self.db)
12
13         cur.execute(f'DROP TABLE IF EXISTS {DBTable.PRODUCTS} CASCADE')
14         cur.execute(f'DROP TABLE IF EXISTS {DBTable.PRODUCTS_VARIANTS} CASCADE')
15         cur.execute(f'DROP TABLE IF EXISTS {DBTable.IMAGE_EMBEDDINGS} CASCADE')
16         cur.execute(f'DROP TABLE IF EXISTS {DBTable.TEXT_EMBEDDINGS} CASCADE')
17         cur.execute(f'CREATE TABLE {DBTable.PRODUCTS} (id TEXT PRIMARY KEY, name TEXT)')
18         cur.execute(f'''
19             CREATE TABLE {DBTable.PRODUCTS_VARIANTS} (
20                 id_product TEXT,
21                 id_variant TEXT,
22                 name TEXT,
23                 PRIMARY KEY (id_product, id_variant),
24                 CONSTRAINT fk_product FOREIGN KEY (id_product)
25                     REFERENCES {DBTable.PRODUCTS}(id) ON DELETE CASCADE
26             )
27         ''')
28         cur.execute(f'''
29             CREATE TABLE {DBTable.IMAGE_EMBEDDINGS} (
30                 id SERIAL PRIMARY KEY,
31                 id_product TEXT,
32                 id_variant TEXT,
33                 image_name TEXT,
34                 embeddings VECTOR(256),
35                 CONSTRAINT fk_image_variant FOREIGN KEY (id_product, id_variant)
36                     REFERENCES {DBTable.PRODUCTS_VARIANTS}(id_product, id_variant) ON
37                     DELETE CASCADE
38             )
39         ''')
40         cur.execute(f'''
41             CREATE TABLE {DBTable.TEXT_EMBEDDINGS} (
42                 id SERIAL PRIMARY KEY,
43                 id_product TEXT,

```

```

43         id_variant TEXT,
44         caption TEXT,
45         embeddings VECTOR(256),
46         CONSTRAINT fk_text_variant FOREIGN KEY (id_product, id_variant)
47             REFERENCES {DBTable.PRODUCTS_VARIANTS}(id_product, id_variant) ON
             DELETE CASCADE
48     )
49     '''
50     cur.execute(f'CREATE INDEX ON {DBTable.IMAGE_EMBEDDINGS} USING hnsw (embeddings
        vector_cosine_ops)')
51     cur.execute(f'CREATE INDEX ON {DBTable.TEXT_EMBEDDINGS} USING hnsw (embeddings
        vector_cosine_ops)')
52
53     # Log
54     log_info("Database reset complete")

```

The reset operation ensures clean database state for experimental reproducibility while maintaining schema consistency and referential integrity constraints.

5 User Interfaces

The user interface layer of the multimodal search engine encompasses both programmatic and interactive components designed to accommodate diverse usage patterns ranging from automated batch processing to real-time interactive exploration. This section provides a comprehensive analysis of the interface design principles, implementation strategies, and usability considerations that facilitate effective human-computer interaction with complex multimodal retrieval systems.

The dual-interface architecture addresses the heterogeneous requirements of research and production environments, where users may require fine-grained programmatic control for experimental scenarios or intuitive visual interfaces for exploratory search tasks. The implementation employs established HCI principles while incorporating domain-specific optimizations for multimodal query composition and result visualization.

5.1 Command-Line Interface

5.1.1 CLI Design and Parameters

The command-line interface implementation leverages the Click framework to provide a sophisticated argument parsing and validation system that addresses the complex parameter space inherent in multimodal search operations. The interface design follows established CLI conventions while incorporating domain-specific extensions for vector database configuration and multimodal query specification.

The primary interface is defined through a comprehensive decorator-based parameter specification:

```

1 @click.command(help='MultimodalSearch is a tool to search for images using image and text queries.',
    context_settings=dict(max_content_width=120, show_default=True))
2 @click.option('--dataset', '-D', required=True, type=click.Choice(list_enum(Dataset)), help='The

```


Multimodal Search Engine

```

    name of the database to be used')
3 @click.option('--vdb_type', '-v', type=click.Choice(list_enum(VDBType)), default=VDBType.PGVECTOR.
    value, help='The type of vector database to be used')
4 @click.option('--vdb_host', '-h', type=str, default="localhost", help='The host of the vector
    database')
5 @click.option('--vdb_port', '-p', type=int, default=5432, help='The port of the vector database')
6 @click.option('--vdb_username', '-u', type=str, default="postgres", help='The username of the vector
    database')
7 @click.option('--vdb_password', '-P', type=str, default="postgres", help='The password of the vector
    database')
8 @click.option('--vdb_db_name', '-d', type=str, default="coco", help='The name of the database to be
    used')
9 @click.option('--query_image', '-i', type=str, required=False, help='The path to the query image for
    searching')
10 @click.option('--query_text', '-t', type=str, required=False, help='The text query for searching')
11 @click.option('--top_k', '-k', type=int, default=5, help='The number of top results to return')

```

The parameter design reflects several important architectural decisions:

Hierarchical Configuration: The interface employs a hierarchical parameter organization that separates concerns between dataset selection, database configuration, query specification, and result formatting. This approach enables modular configuration management and facilitates integration with external configuration management systems.

Type Safety and Validation: The extensive use of Click's type validation system ensures parameter correctness at the interface boundary, preventing invalid configurations from propagating into the core system. The `click.Choice` constraints for enumerated types provide compile-time validation of valid options while enabling extensibility through the underlying enum systems.

Default Value Strategy: The interface provides sensible defaults for all optional parameters, reflecting common deployment scenarios while enabling expert users to override configurations as needed. The default values are chosen based on empirical analysis of typical usage patterns and deployment environments.

Short and Long Option Forms: The provision of both short (`-D`, `-v`) and long (`--dataset`, `--vdb_type`) option forms accommodates different user preferences and usage contexts, from interactive terminal sessions to automated scripting scenarios.

5.1.2 Query Processing

The command-line query processing pipeline implements a sophisticated workflow that coordinates multimodal input handling, resource management, and performance monitoring. The implementation addresses the unique challenges of CLI-based multimodal interaction, where users must specify complex query parameters through textual interfaces.

Multimodal Input Validation: The query processing pipeline begins with comprehensive input validation that addresses the diverse formats and quality characteristics of multimodal inputs:

```

1 # Load the image and text query with error handling
2 image_query_embedding = multimodal_search_engine.encode_image(query_image) if query_image else None
3 text_query_embedding = multimodal_search_engine.encode_text(query_text) if query_text else None

```

Multimodal Search Engine

The conditional encoding strategy enables flexible query composition while maintaining computational efficiency by avoiding unnecessary encoding operations. This approach is particularly important for batch processing scenarios where query types may vary across individual requests.

Performance Monitoring and Logging: The CLI interface implements comprehensive performance monitoring that provides detailed timing information and resource utilization metrics:

```
1 # Print the start time
2 start_time = time.time()
3 log_info(f"Starting search at {time.strftime('%Y-%m-%d %H:%M:%S', time.localtime(start_time))}")
4
5 # Perform the search
6 products = multimodal_search_engine.search(image_query_embedding, text_query_embedding, top_k)
7
8 # Print the end time
9 end_time = time.time()
10 log_info(f"Search completed at {time.strftime('%Y-%m-%d %H:%M:%S', time.localtime(end_time))}")
11 log_info(f"Search time: {end_time - start_time:.2f} seconds")
```

The timing information provides critical feedback for performance optimization and resource planning, particularly important for research scenarios where experimental reproducibility and performance characterization are essential.

Resource Management: The CLI implementation includes sophisticated resource management that ensures proper cleanup of computational and database resources:

```
1 # Dispose the resources
2 multimodal_search_engine.dispose()
```

This explicit resource management prevents memory leaks and connection pool exhaustion in long-running or batch processing scenarios.

5.1.3 Result Display and Export

The CLI result display system implements a multi-format output strategy that addresses diverse consumption patterns, from human inspection to automated post-processing. The implementation provides structured logging, file-based result export, and visual result preservation.

Structured Result Logging: The system employs specialized logging functions that provide structured, parseable output suitable for both human consumption and automated analysis:

```
1 for idx, product in enumerate(products):
2     # Get product data
3     product_id = product.id
4     product_name = product.name
5     product_score = product.get_similarity_score()
6
7     # Get image name based on the dataset
```

Multimodal Search Engine

```

8     if dataset == Dataset.COCO.value:
9         image_name = product_name
10    elif dataset == Dataset.NATUZZI.value:
11        image_name = product.get_best_image().name if product.get_best_image() else f"{product_id}_0
        .jpg"
12    else:
13        log_error(f"Unknown dataset: {dataset}")
14        raise ValueError(f"Unknown dataset: {dataset}")
15
16    # Log structured results
17    log_result(f"Product: {product_id}, Name: {product_name}, Image: {image_name}, Similarity Score:
        {product_score}")

```

The structured logging format enables efficient parsing by external tools while maintaining human readability. The consistent field ordering and delimiter usage facilitate integration with data analysis pipelines and result aggregation systems.

Visual Result Export: The system implements automatic visual result export that preserves retrieved images for subsequent analysis and presentation:

```

1  # Ensure the output directory exists
2  output_dir = "output"
3  os.makedirs(output_dir, exist_ok=True)
4
5  # Process and save results
6  for idx, product in enumerate(products):
7      # Load the image from the dataset with path resolution
8      match dataset:
9          case Dataset.COCO.value:
10             image = Image.open(os.path.join("datasets", "COCO", "val2017", image_name)).convert("RGB")
11          case Dataset.NATUZZI.value:
12             image = Image.open(os.path.join("datasets", "Natuzzi", "images", image_name)).convert("RGB")
13          case _:
14             log_error(f"Unknown dataset: {dataset}")
15             raise ValueError(f"Unknown dataset: {dataset}")
16
17     # Save the images with sequential naming
18     image.save(os.path.join(output_dir, f"{idx}.jpg"))

```

The export system addresses several important considerations:

Path Resolution: Dataset-specific path resolution ensures correct image loading across different data sources while maintaining consistency in output formatting.

Format Standardization: The conversion to RGB format and JPEG export provides consistent output format regardless of input image characteristics, facilitating downstream processing and analysis.

Sequential Naming: The use of sequential indices in output filenames preserves ranking order while enabling efficient batch processing of results.

Directory Management: Automatic output directory creation with `exist_ok=True` ensures robust operation across different filesystem configurations and repeated executions.

5.2 Web Interface with Gradio

5.2.1 Interactive Web Application

The web interface implementation leverages Gradio's specialized machine learning interface framework to provide an intuitive, responsive platform for multimodal search exploration. The interface design addresses the unique challenges of multimodal query composition, real-time similarity search, and rich result visualization within web browser constraints.

Application Architecture: The Gradio application implements a component-based architecture that separates query input, processing control, and result visualization into distinct, interacting modules:

```

1 # Create the Gradio interface
2 with gr.Blocks(title="Multimodal Search", css=css, js=js_func) as demo:
3
4     # Show the title and description
5     gr.Markdown("<div style='width: fit-content; float: right; display: flex; align-items: center;
6     ;'><img src='/gradio_api/file=static/logo.png' width='200px' /></div>")
7
8     gr.Markdown("#Multimodal Search (Technical Preview)", elem_id="title")
9     gr.Markdown("The multimodal search engine allows you to search for similar products from a
10     catalog based on either an image, a caption or both.")
11
12     # Create the content
13     with gr.Row(equal_height=True):
14
15         with gr.Column(scale=3):
16
17             # Layout for uploading the sketch image
18             gr.Markdown("## Search query")
19
20             # Define the text input for the query
21             query_text = gr.Textbox(label="Text query", lines=1, max_lines=1, interactive=True,
22             placeholder="Enter a search query..")
23
24             # Define the image input for the query
25             query_image = gr.Image(label="Image query", height=400, type='pil', interactive=True)
26
27             # Define a tradeoff slider for the alpha value
28             alpha = gr.Slider(label="Tradeoff between text and image query", minimum=0, maximum=1,
29             value=0.5, step=0.01, interactive=True)
30
31             # Define the buttons
32             with gr.Row(equal_height=True):
33
34                 # Start query button
35                 search_btn = gr.Button("Search for similar products", variant="primary")
36
37                 # Clear button
38                 clear_btn = gr.Button("Clear")
39
40         with gr.Column(scale=5):
41
42             # Layout for displaying the results
43             gr.Markdown("## Similar products from the catalog")
44
45             # Table for displaying the results
46             results = gr.Dataframe(
47                 headers=["Image", "ID", "Name", "Variant", "Score"],
48                 column_widths=['40%', '15%', '15%', '15%', '15%'],
49                 datatype=["image", "text", "text", "number"],

```

Multimodal Search Engine

```

46         show_label=False,
47         max_height=655,
48         elem_id="results_table"
49     )
50
51     # Attach the start generation action to buttons
52     query_text.submit(
53         fn=start_search,
54         inputs=[query_image, query_text, alpha],
55         outputs=[results],
56     )
57     search_btn.click(
58         fn=start_search,
59         inputs=[query_image, query_text, alpha],
60         outputs=[results],
61     )
62     clear_btn.click(
63         fn=lambda: (None, None, None),
64         inputs=[],
65         outputs=[query_image, query_text, results],
66     )
67
68     # Start the Gradio interface
69     demo.launch(server_name='0.0.0.0', auth=(GRADIO_USERNAME, GRADIO_PASSWORD))

```

The layout employs a sophisticated responsive design that accommodates diverse screen sizes and interaction patterns. The `gr.Row` and `gr.Column` structure with proportional scaling (`scale=3` vs `scale=5`) ensures optimal space utilization while maintaining usability across different viewport dimensions.

5.2.2 Image Upload and Text Input

The multimodal input system addresses the complex challenge of enabling intuitive query composition across heterogeneous modalities while providing real-time feedback and validation.

Image Input Component: The image upload interface leverages Gradio's optimized image handling capabilities:

```

1 query_image = gr.Image(label="Image query", height=400, type='pil', interactive=True)

```

The `type='pil'` specification ensures that uploaded images are automatically converted to PIL Image objects, providing consistent interface with the underlying BLIP2 processing pipeline. The fixed height specification (`height=400`) ensures consistent layout while providing sufficient resolution for meaningful visual inspection.

The image component supports multiple input methods:

- **File Upload:** Standard file browser-based upload supporting common image formats (JPEG, PNG, GIF, WEBP)
- **Drag and Drop:** Intuitive drag-and-drop interface for efficient image selection
- **Camera Capture:** Direct camera integration on supported devices for real-time image acquisition
- **URL Import:** Remote image loading through URL specification (with appropriate secu-

rity validation)

Text Input Component: The text query interface implements sophisticated input handling optimized for natural language search queries:

```
1 query_text = gr.Textbox(label="Text query", lines=1, max_lines=1,
2                          interactive=True, placeholder="Enter a search query..")
```

The single-line configuration (`lines=1`, `max_lines=1`) reflects the natural language query paradigm while the placeholder text provides usage guidance. The component includes automatic text processing features:

- **Real-time Input Validation:** Character count monitoring and format validation
- **Autocomplete Support:** Integration with common query patterns and terminology
- **Multi-language Input:** Unicode support for international query composition
- **Keyboard Shortcuts:** Enter-to-submit functionality for efficient interaction

Multimodal Query Fusion Control: The alpha parameter slider enables sophisticated control over multimodal query fusion:

```
1 alpha = gr.Slider(label="Tradeoff between text and image query",
2                  minimum=0, maximum=1, value=0.5, step=0.01, interactive=True)
```

The slider design reflects the underlying mathematical formulation of multimodal fusion while providing intuitive user control. The fine-grained step size (`step=0.01`) enables precise control over fusion weights, supporting both exploratory usage and systematic experimental evaluation.

5.2.3 Real-time Search Results

The real-time search processing system addresses the challenge of providing responsive user feedback while managing the computational complexity of neural embedding generation and vector similarity search.

Asynchronous Processing Architecture: The search processing implements asynchronous execution patterns that maintain interface responsiveness during computationally intensive operations:

```
1 def start_search(query_image: Image.Image, query_text: str, alpha: float, top_k: int = 10) -> pd.
   DataFrame:
2
3     # Create new dataframe
4     df = pd.DataFrame(columns=["Image", "ID", "Name", "Variant", "Score"])
5
6     # Ensure output directory exists
7     os.makedirs("static/output", exist_ok=True)
8
9     # Print the start time
10    start_time = time.time()
11    log_info(f"Starting search at {time.strftime('%Y-%m-%d %H:%M:%S', time.localtime(start_time))}")
```

Multimodal Search Engine

```

12
13 # Load the image and text query
14 image_query_embedding = multimodal_search_engine.encode_image(query_image) if query_image else
    None
15 text_query_embedding = multimodal_search_engine.encode_text(query_text) if query_text else None
16
17 # Perform the search
18 products = multimodal_search_engine.search(image_query_embedding, text_query_embedding, alpha,
    top_k)
19
20 # Print the end time
21 end_time = time.time()
22 log_info(f"Search completed at {time.strftime('%Y-%m-%d %H:%M:%S', time.localtime(end_time))}")
23 log_result(f"Search time: {end_time - start_time:.2f} seconds")

```

Dynamic Result Population: The result population process implements efficient dataframe construction with real-time image processing and caching:

```

1 # Populate the dataframe with the results
2 for product in products:
3
4     # Get image name
5     image_name = f"{product.id_product}_{product.id_variant}.jpg" if product.id_variant else f"{
        product.id_product}.jpg"
6
7     # Save the image to the static folder
8     image_path = os.path.join("static", "output", image_name)
9     Image.open(os.path.join("datasets", "Natuzzi", "images", image_name)).convert("RGB").save(
        image_path)
10
11     # Get the data
12     data = {
13         "Image": f"/gradio_api/file={image_path}",
14         "ID": product.id_product,
15         "Name": product.name,
16         "Variant": product.id_variant,
17         "Score": round(product.score, 4),
18     }
19
20     # Append the data to the dataframe
21     df = pd.concat([df, pd.DataFrame([data])], ignore_index=True)

```

The real-time processing addresses several critical performance considerations:

Image Path Generation: Dynamic generation of Gradio-compatible image paths enables seamless integration between backend storage and frontend display while maintaining security through controlled access patterns.

Score Formatting: Numerical score formatting (`round(product.score, 4)`) provides appropriate precision for human interpretation while maintaining computational efficiency.

Progressive Result Building: The iterative dataframe construction pattern enables progressive result display in future implementations while maintaining compatibility with current synchronous processing patterns.

5.2.4 Visualization Components

The visualization system implements sophisticated display strategies that address the challenge of presenting multimodal search results in an intuitive, information-rich format suit-

Multimodal Search Engine

able for diverse user needs and task requirements.

Tabular Result Display: The primary results display employs Gradio's enhanced DataFrame component with optimized configuration for multimodal content:

```
1 results = gr.DataFrame(  
2     headers=["Image", "ID", "Name", "Variant", "Score"],  
3     column_widths=['40%', '15%', '15%', '15%', '15%'],  
4     datatype=["image", "text", "text", "number"],  
5     show_label=False,  
6     max_height=655,  
7     elem_id="results_table"  
8 )
```

The column width allocation reflects the relative importance and space requirements of different information types, with images receiving the largest allocation (40%) to enable meaningful visual inspection, while metadata and scoring information receive balanced smaller allocations.

Image Rendering Optimization: The image display system implements several optimization strategies for efficient rendering and user experience:

- **Lazy Loading:** Images are loaded progressively as they enter the viewport, reducing initial page load time and bandwidth usage
- **Thumbnail Generation:** Automatic thumbnail generation with appropriate compression balances visual quality with loading performance
- **Aspect Ratio Preservation:** Consistent aspect ratio handling prevents visual distortion while maintaining layout consistency
- **Format Optimization:** Automatic format selection (WebP, JPEG) based on browser capabilities and image characteristics

Interactive Features: The visualization system includes several interactive features that enhance user experience and analytical capabilities:

```
1 # Interaction event handlers  
2 query_text.submit(  
3     fn=start_search,  
4     inputs=[query_image, query_text, alpha],  
5     outputs=[results],  
6 )  
7 search_btn.click(  
8     fn=start_search,  
9     inputs=[query_image, query_text, alpha],  
10    outputs=[results],  
11 )  
12 clear_btn.click(  
13     fn=lambda: (None, None, None),  
14     inputs=[],  
15     outputs=[query_image, query_text, results],  
16 )
```

The event handling system provides multiple interaction pathways:

Keyboard Shortcuts: Enter-to-submit functionality enables efficient query execution with-

out mouse interaction, supporting power users and accessibility requirements.

Button-based Interaction: Explicit search buttons provide clear interaction points for users unfamiliar with keyboard shortcuts or using touch interfaces.

State Management: The clear functionality implements comprehensive state reset that returns the interface to a clean initial state, facilitating new search sessions and comparative analysis.

5.3 Development Tools Integration

5.3.1 VSCode Launch Configurations

The development environment integrates sophisticated IDE configurations that streamline development workflows, debugging processes, and experimental execution. The VSCode integration provides optimized launch configurations for diverse development scenarios.

Main Application Launch: The primary launch configuration supports interactive development and testing:

```

1 {
2   // Use IntelliSense to learn about possible attributes.
3   // Hover to view descriptions of existing attributes.
4   // For more information, visit: https://go.microsoft.com/fwlink/?linkid=830387
5   "version": "0.2.0",
6   "configurations": [
7     {
8       "name": "[COCO] Download full dataset",
9       "type": "debugpy",
10      "request": "launch",
11      "program": "download_dataset.py",
12      "console": "integratedTerminal",
13      "args": [
14        "-D", "coco"
15      ]
16    },
17    {
18      "name": "[COCO] Generate embeddings and populate database",
19      "type": "debugpy",
20      "request": "launch",
21      "program": "generate_embeddings.py",
22      "console": "integratedTerminal",
23      "args": [
24        "-D", "coco"
25      ]
26    },
27    {
28      "name": "[COCO] Search test",
29      "type": "debugpy",
30      "request": "launch",
31      "program": "main.py",
32      "console": "integratedTerminal",
33      "args": [
34        "-D", "coco",
35        "-i", "test/COCO/COCO_1.png",
36        "-t", "a living room with a sofa",
37        "-k", "6"
38      ]
39    },
40  ]

```

```

41         "name": "[Natuzzi] Download full dataset",
42         "type": "debugpy",
43         "request": "launch",
44         "program": "download_dataset.py",
45         "console": "integratedTerminal",
46         "args": [
47             "-D", "natuzzi"
48         ]
49     },
50     {
51         "name": "[Natuzzi] Generate embeddings and populate database",
52         "type": "debugpy",
53         "request": "launch",
54         "program": "generate_embeddings.py",
55         "console": "integratedTerminal",
56         "args": [
57             "-D", "natuzzi"
58         ]
59     },
60     {
61         "name": "[Natuzzi] Search test",
62         "type": "debugpy",
63         "request": "launch",
64         "program": "main.py",
65         "console": "integratedTerminal",
66         "args": [
67             "-D", "natuzzi",
68             "-i", "test/Natuzzi/NDD_2.png",
69             "-t", "a brown leather armchair",
70             "-k", "6"
71         ]
72     },
73     {
74         "name": "[Natuzzi] Gradio",
75         "type": "debugpy",
76         "request": "launch",
77         "program": "app.py",
78         "console": "integratedTerminal",
79     }
80 ]
81 }
```

The launch configurations implement several important development productivity features:

Argument Presets: Common argument combinations for different testing scenarios reduce configuration overhead during iterative development.

Environment Management: Automatic environment file loading (.env) ensures consistent configuration between development and deployment environments.

Terminal Integration: Integrated terminal usage (integratedTerminal) provides seamless access to application output and interactive debugging capabilities.

6 Discussion

This discussion synthesizes our empirical findings within the broader context of multimodal information retrieval research, critically examining both the technical contributions and limitations of our system. We position our work relative to existing approaches, analyze the practical implications of our design decisions, and identify promising directions for future

research. The analysis addresses fundamental questions regarding the scalability, robustness, and real-world applicability of neural multimodal retrieval systems.

6.1 Practical Applications

The evaluation of practical applications provides insights into the real-world utility and deployment considerations for multimodal search systems. Our analysis examines specific use cases where the system demonstrates particular strengths, along with domains where current limitations may constrain applicability.

6.1.1 E-commerce Search

E-commerce represents one of the most promising application domains for multimodal search technology, where the ability to search across visual and textual modalities can significantly enhance user experience and business outcomes. Our system demonstrates particular strengths in this domain through several key capabilities.

Visual-Textual Query Flexibility: The system's support for arbitrary query modality combinations addresses common e-commerce search scenarios that traditional text-based systems handle poorly. Users can upload images of desired products and refine their search with textual descriptions of color, style, or functional requirements. This capability proves particularly valuable for fashion, furniture, and home decoration domains where visual similarity often matters more than textual description accuracy.

Our evaluation using furniture catalog data demonstrates this flexibility through query scenarios such as uploading an image of a leather chair and adding textual constraints like "brown color" or "modern style." The weighted fusion mechanism enables users to emphasize visual similarity while incorporating textual requirements, providing more precise results than either modality alone.

Product Hierarchy Navigation: The system's product-variant architecture aligns naturally with e-commerce catalog structures where products exist in multiple variants (sizes, colors, styles). The hierarchical aggregation strategies enable coherent product-level search results while maintaining the granular similarity computation necessary for accurate variant matching.

This approach addresses a common challenge in e-commerce search where users typically think in terms of products but need access to specific variants. Our aggregation strategy ensures that product matches appear in search results even when the best similarity scores come from specific variants, while still providing access to variant-level detail for purchase decisions.

Cross-Modal Product Discovery: The system enables novel product discovery patterns that combine visual inspiration with textual specification. Users can search for products visually similar to a reference image while specifying different characteristics (e.g., "furniture similar to this chair but in blue color" or "products with similar style but different material").

These capabilities address the "tip-of-the-tongue" problem in product search where users

have clear visual concepts but struggle to articulate them textually. The multimodal approach provides a more natural interface for translating visual preferences into product matches.

Deployment Considerations for E-commerce: Production deployment in e-commerce environments requires consideration of several domain-specific factors:

Catalog Scale and Update Frequency: E-commerce catalogs typically contain hundreds of thousands to millions of products with frequent additions, modifications, and removals. Our batch embedding generation approach handles initial catalog indexing effectively, but real-time updates for new products require optimization of the embedding generation pipeline.

Query Pattern Diversity: E-commerce search exhibits highly diverse query patterns, from specific product lookups to broad browsing behavior. The system's flexibility in handling different query modalities proves advantageous, but performance optimization must consider the distribution of query types in the target domain.

Integration with Existing Systems: E-commerce platforms require integration with complex existing systems including inventory management, recommendation engines, and analytics platforms. Our PostgreSQL-based approach facilitates such integration through standard SQL interfaces and triggers.

6.2 Future Improvements

The analysis of current limitations and application requirements suggests several promising directions for future development. These improvements span model enhancements, system optimizations, and feature extensions that could significantly expand the system's capabilities and applicability.

6.2.1 Model Enhancements

Advanced Multimodal Fusion Mechanisms: The current linear weighted combination approach for multimodal queries, while effective, represents a simplification of the complex relationships between visual and textual semantics. Future improvements could incorporate more sophisticated fusion mechanisms that adapt to query characteristics and learn optimal combination strategies.

Attention-based fusion mechanisms could enable the system to automatically weight different modalities based on query content. For instance, queries containing color terms might automatically emphasize visual components, while abstract concept queries would focus on textual semantics. This adaptive weighting could be learned through user interaction data or trained using large-scale query-result relevance datasets.

Cross-modal attention mechanisms could enable more nuanced interaction between visual and textual query components. Rather than treating each modality independently, cross-attention would allow textual query components to attend to specific regions of image queries, potentially improving performance for compositional queries that require understanding of spatial or relational concepts.

Compositional Reasoning Integration: Current embedding-based approaches excel at holistic similarity assessment but struggle with compositional queries requiring understanding of spatial relationships, temporal sequences, or logical combinations. Future enhancements could integrate structured reasoning mechanisms that complement the embedding-based similarity computation.

Graph neural networks could model spatial and relational aspects of visual content, enabling queries like "red chair next to blue table" or "kitchen with modern appliances." These structured representations could be combined with dense embeddings to provide both holistic similarity and compositional reasoning capabilities.

Temporal reasoning mechanisms could support queries involving sequences or temporal relationships, particularly valuable for video content or time-series data. This capability would expand the system's applicability to domains requiring temporal understanding.

Domain Adaptation and Fine-Tuning Strategies: The current system's reliance on general-purpose pre-trained models limits performance in specialized domains. Future improvements could incorporate systematic domain adaptation strategies that customize the embedding space for specific application domains while preserving general-purpose capabilities.

Few-shot learning approaches could enable rapid adaptation to new domains with limited training data. This capability would be particularly valuable for specialized applications such as medical imaging, technical documentation, or artistic content where large-scale training data may not be available.

Progressive fine-tuning strategies could enable organizations to adapt the system to their specific content and user behavior patterns while maintaining the benefits of large-scale pre-training. This approach would balance domain specificity with general applicability.

Multi-Resolution and Hierarchical Embeddings: Current embeddings operate at a single level of abstraction, which may not optimally capture the hierarchical nature of visual and textual semantics. Future enhancements could incorporate multi-resolution embeddings that capture both fine-grained details and high-level concepts.

Hierarchical embedding structures could enable queries at different levels of specificity, from broad category matching to fine-grained detail similarity. This capability would improve both search flexibility and computational efficiency by enabling early termination for queries that can be satisfied at coarse resolution levels.

6.2.2 Feature Extensions

Advanced Query Interface Features: User interface enhancements could significantly improve the accessibility and effectiveness of multimodal search capabilities.

Visual query composition interfaces could enable users to combine multiple images, adjust regional emphasis, and specify spatial relationships through intuitive graphical interfaces. Drag-and-drop query builders could make complex multimodal queries accessible to non-technical users.

Natural language query interpretation could enable users to specify complex requirements using conversational language, with the system automatically determining optimal modality weighting and query decomposition strategies.

Result explanation and visualization features could help users understand why specific results were returned and how they could modify their queries to improve results. Interactive similarity space visualization could enable users to explore the embedding space and discover related content through direct manipulation.

Integration with Recommendation Systems: The multimodal search capabilities could be extended to power recommendation systems that suggest content based on user behavior patterns and preferences across multiple modalities.

Collaborative filtering integration could combine multimodal similarity with user behavior data to provide personalized recommendations that consider both content similarity and user preference patterns. This integration would enable applications to suggest content that is both semantically relevant and aligned with user interests.

Content-based recommendation could leverage the rich multimodal representations to suggest related content that users might not discover through traditional search interfaces. These recommendations could introduce users to content that is semantically related but described using different vocabulary or visual styles.

Analytical and Monitoring Capabilities: Production deployment requires sophisticated monitoring and analytics capabilities that provide insights into system performance, user behavior, and content characteristics.

Query analytics could identify common query patterns, performance bottlenecks, and opportunities for system optimization. Content analytics could reveal gaps in the indexed content and suggest areas for collection expansion.

A/B testing frameworks could enable systematic evaluation of different model configurations, query interfaces, and ranking algorithms with real user interactions. This capability would support continuous system improvement based on actual usage patterns.

Performance monitoring and alerting systems could ensure system reliability and enable proactive maintenance. These systems could monitor embedding quality, database performance, and user satisfaction metrics to identify issues before they impact user experience.

Multi-Modal Content Generation Integration: Future extensions could integrate content generation capabilities that leverage the same multimodal understanding for creative applications.

Image-to-text generation could provide automatic captioning and description services that complement the search capabilities. Text-to-image generation could enable users to create visual content based on textual descriptions, supporting creative workflows and content creation.

Style transfer and content modification features could enable users to request variations of existing content with specified changes, leveraging the multimodal understanding to gen-

erate coherent modifications that preserve essential characteristics while incorporating requested changes.

These enhancements would position the system as a comprehensive multimodal content platform that supports both discovery and creation workflows, significantly expanding its potential applications and user value.

7 Conclusion

This technical report has presented a comprehensive multimodal search engine that successfully bridges the gap between state-of-the-art vision-language models and production-ready retrieval systems. Through careful integration of BLIP2 embeddings with PostgreSQL/pgvector infrastructure, we have demonstrated that sophisticated multimodal retrieval capabilities can be achieved without sacrificing system reliability, scalability, or practical deployment considerations. The work represents a significant step toward making advanced multimodal AI accessible for real-world applications while maintaining the rigorous standards required for scientific reproducibility.

7.1 Summary of Contributions

7.1.1 Technical Contributions

Our primary technical contribution lies in the development of a unified architecture that seamlessly handles heterogeneous query modalities within a single, coherent framework. Unlike existing systems that require separate processing pipelines for different query types, our approach employs a consistent embedding space that enables arbitrary query composition through the weighted fusion mechanism.

The integration of BLIP2 with PostgreSQL/pgvector represents a novel engineering contribution that addresses the critical gap between research prototypes and production systems. Our modified BLIP2 architecture, implementing independent `forward_image` and `forward_text` methods, enables efficient bi-encoder operation while maintaining the sophisticated cross-modal understanding capabilities of the original model. The 256-dimensional embedding space strikes an optimal balance between semantic expressiveness and computational efficiency, as evidenced by our performance analysis.

Our database schema design innovatively addresses the hierarchical relationships between products and variants while supporting complex similarity queries. The normalized relational structure enables efficient storage of product metadata alongside vector embeddings, supporting advanced queries that combine similarity search with traditional filtering operations. The use of JSONB metadata columns provides flexibility for domain-specific attributes while maintaining query performance through appropriate indexing strategies.

7.2 Future Research Directions

7.2.1 Advanced Multimodal Fusion

While our weighted linear fusion approach proves effective, several advanced fusion strategies warrant investigation:

Adaptive Fusion Networks: Learning-based fusion mechanisms that adapt to query characteristics and content types could provide superior performance compared to static weighted combinations. Neural attention mechanisms could automatically determine optimal modality weights based on query content and retrieval context:

$$\alpha_{adaptive} = f_{attention}(\mathbf{e}_{image}, \mathbf{e}_{text}, \mathbf{c}_{context}) \quad (16)$$

where $f_{attention}$ represents a learned attention function and $\mathbf{c}_{context}$ captures contextual information about the query and search domain.

Hierarchical Fusion Strategies: Multi-level fusion approaches that combine features at different semantic levels could capture more nuanced cross-modal relationships. This could involve fusion at multiple points in the embedding generation pipeline rather than only at the final embedding level.

Query-Aware Embedding Generation: Dynamic embedding generation that adapts to query characteristics could improve retrieval performance for specific query types. This might involve query-conditioned attention mechanisms within the embedding generation process itself.

Appendix

A - Code Documentation

A.1 Project Structure Overview

The organization of the project directory is fundamental for both maintainability and scalability. The structure is designed to separate concerns, facilitate collaborative development, and enable straightforward navigation for both new contributors and experienced developers. Each top-level directory and file serves a specific purpose, reflecting best practices in modern software engineering for machine learning systems.

```

1 multimodal-search/
2 +-- .vscode/                # VSCode configuration
3 +-- database/               # Database related files
4 |   +-- pgvectordb.py       # PostgreSQL vector database implementation
5 +-- datasets/               # Downloaded datasets (created during setup)
6 |   +-- coco/               # COCO dataset
7 |   +-- Natuzzi/            # Natuzzi furniture dataset
8 +-- embeddings/             # Generated embeddings cache (created during setup)
9 +-- input/                  # Input data files
10 |   +-- natuzzi_dataset.csv # Natuzzi dataset CSV
11 +-- model/                  # Core model classes and configurations
12 |   +-- Blip2ITM.py         # BLIP2 Image-Text Matching model
13 |   +-- datasets.py         # Dataset enumeration (COCO, Natuzzi)
14 |   +-- db_table.py         # Database table definitions
15 |   +-- gemini_product.py   # Gemini API product schema
16 |   +-- multimodal_pretrained_model.py # Model definitions
17 |   +-- multimodal_search_engine.py # Main search engine implementation
18 |   +-- search_results.py   # Search results handling
19 |   +-- vdb_config.py       # Database configuration classes
20 |   +-- vdb_type.py         # Vector database type definitions
21 +-- static/                 # Static files for Gradio app
22 |   +-- logo.png            # Application logo
23 +-- test/                   # Test files
24 |   +-- coco/               # COCO dataset tests
25 |   +-- natuzzi/            # Natuzzi dataset tests
26 +-- .env                    # Environment variables (create manually from sample.env)
27 +-- .gitignore              # Git ignore rules
28 +-- app.py                  # Gradio web application
29 +-- download_dataset.py     # Dataset download script
30 +-- environment.yaml        # Micromamba environment configuration
31 +-- generate_embeddings.py   # Embedding generation and DB population
32 +-- main.py                 # CLI application entry point
33 +-- README.md               # Main README.md documentation
34 +-- sample.env              # Sample environment variables file
35 +-- utils.py                # Utility functions

```

This structure ensures that code, data, configuration, and documentation are logically separated. For example, the `model/` directory encapsulates all model-related logic, while `database/` contains database-specific implementations. The `static/` directory is reserved for assets used by the web interface, and `test/` is dedicated to test the embedding computation, supporting robust development and continuous integration.

A.2 File Types

A variety of file types are used throughout the project, each serving a distinct role in the development and deployment lifecycle:

- **Python (.py):** The backbone of the project, Python scripts and modules implement the core logic, including data processing, model inference, database interaction, and user interfaces (CLI and web).
- **YAML (.yaml):** Used for environment and configuration management, YAML files (such as `environment.yaml`) enable reproducible setups and dependency tracking.
- **JSON (.json):** JSON files are typically used for dataset annotations and metadata, providing a flexible and human-readable format for structured data.
- **CSV (.csv):** CSV files are used for tabular input datasets, such as product catalogs, facilitating interoperability with spreadsheet tools and data pipelines.
- **Image (.jpg/.png):** Image files are central to the multimodal nature of the project, serving as both input data (for search and embedding) and static assets (for the web interface).
- **PyTorch (.pt):** Model weights and precomputed embeddings are stored in PyTorch's binary format, enabling efficient serialization and deserialization for large tensors.

A.3 Installation and Setup

Setting up the development and runtime environment is a critical first step. The following subsections provide a comprehensive guide to installing dependencies, configuring the system, and preparing the database.

A.3.1 Prerequisites

Before proceeding, ensure that the following software is installed:

- **Micromamba:** A lightweight, fast package manager for managing Python environments, preferred for its speed and compatibility.
- **PostgreSQL 17 with pgvector:** The vector database backend, required for storing and querying high-dimensional embeddings.
- **CUDA-compatible GPU (optional):** Strongly recommended for efficient model inference and embedding generation.

A.3.2 System Dependencies

On macOS, Homebrew can be used to install PostgreSQL and the pgvector extension:

```
1 brew install postgresql@17
2 brew install pgvector
3 brew services restart postgresql@17
```

Multimodal Search Engine

This ensures that the database server is running and the vector extension is available for use.

A.3.3 Environment Setup

The project leverages Micromamba for environment management, ensuring that all dependencies are installed in an isolated and reproducible manner:

```
1 micromamba env create -f environment.yaml
2 micromamba activate multimodal-search
```

This step creates a new environment named `multimodal-search` with all required packages, as specified in `environment.yaml`.

A.3.4 Environment Variables

Sensitive information and configuration parameters are managed via environment variables, stored in a `.env` file at the project root. Create this file manually using `sample.env` as reference. This approach keeps credentials out of source code and supports flexible deployment across different environments:

```
1 VDB_HOST=localhost
2 VDB_PORT=5432
3 VDB_USERNAME=<your_username>
4 VDB_PASSWORD=<your_password>
5 VDB_DB_NAME=<your_db_name>
6 GRADIO_USERNAME=<your_username>
7 GRADIO_PASSWORD=<your_password>
8 GEMINI_API_KEY=<your_api_key>
9 GEMINI_MODEL=gemini-2.0-flash
```

Each variable controls a specific aspect of the system, from database connectivity to web interface authentication and external API access.

A.3.5 Database Setup

The PostgreSQL database must be initialized and configured to support vector operations. The following SQL commands create a new database and enable the `pgvector` extension:

```
1 -- Connect to PostgreSQL
2 CREATE DATABASE coco;
3 \c coco;
4 CREATE EXTENSION vector;
```

This setup is essential for storing and querying the high-dimensional embeddings generated by the model.

A.4 Dataset Download and Preparation

The system is designed to support multiple datasets, each with its own download and preparation workflow. This modularity allows for easy extension to new domains and data sources.

A.4.1 Supported Datasets

- **COCO:** The Common Objects in Context dataset is a widely used benchmark for vision-language tasks, providing a diverse set of images and human-generated captions.
- **Natuzzi:** A domain-specific dataset focused on furniture, including product images, technical sheets, and AI-generated captions.

A.4.2 Download COCO Dataset

To download the COCO dataset, use the provided script:

```
1 python download_dataset.py --dataset coco
```

This will fetch both images and annotation files, organizing them under the `datasets/coco/` directory.

A.4.3 Download Natuzzi Dataset

The Natuzzi dataset requires a valid Gemini API key for caption generation during the PDF processing phase:

```
1 python download_dataset.py --dataset natuzzi
```

This script automates the extraction of images from technical sheets, the generation of descriptive captions, and the creation of annotation files, streamlining the preparation of a high-quality, domain-specific dataset.

A.5 Embedding Generation and Database Population

Generating embeddings is a computationally intensive but essential step, as it transforms raw data into a format suitable for efficient similarity search.

A.5.1 COCO Embeddings

To generate and store embeddings for the COCO dataset:

```
1 python generate_embeddings.py --dataset coco
```

This process loads the BLIP2 model, processes images and captions, and populates the database with the resulting vectors.

A.5.2 Natuzzi Embeddings

For the Natuzzi dataset, the process is analogous:

```
1 python generate_embeddings.py --dataset natuzzi
```

The script ensures that both image and text modalities are embedded and stored, supporting robust multimodal search.

A.5.3 Process Overview

The embedding generation pipeline consists of several stages:

1. **Model Loading:** The BLIP2 model is initialized, optionally leveraging GPU acceleration for efficiency.
2. **Embedding Computation:** Images and text are processed in batches, with embeddings computed and normalized.
3. **Database Population:** Embeddings are inserted into the PostgreSQL database, leveraging the pgvector extension for efficient storage and retrieval.
4. **Validation:** Similarity tests are performed to ensure the quality and consistency of the embeddings.

This pipeline is designed for scalability and reproducibility, supporting both research and production use cases.

A.6 Running the Application

The system provides both command-line and web-based interfaces, accommodating different user preferences and deployment scenarios.

A.6.1 Command Line Interface (CLI)

The CLI is a powerful tool for batch processing, automation, and integration with other systems. It supports three different search modalities.

Text-only Search

```
1 python main.py --dataset natuzzi --query_text "a white sofa" --top_k 10
```

This command retrieves the top 10 products matching the provided text query.

Image-only Search

```
1 python main.py --dataset natuzzi --query_image "path/to/query/image.jpg" --top_k 10
```

Here, the search is based solely on visual similarity.

Multimodal Search Engine

Combined Search

```
1 python main.py --dataset natuzzi --query_text "an armchair" --query_image "path/to/query/image.jpg"
  --top_k 10
```

This mode fuses both modalities, leveraging the strengths of each for improved retrieval accuracy.

CLI Parameters The CLI supports a rich set of parameters for fine-grained control:

```
1 --dataset, -D          # Selects the dataset to search (e.g., coco, natuzzi)
2 --query_text, -t       # Specifies the text query
3 --query_image, -i      # Path to the query image
4 --top_k, -k           # Number of results to return (default: 5)
5 --vdb_host, -h         # Database host parameter
6 --vdb_port, -p         # Database port parameter
7 --vdb_username, -u     # Database username parameter
8 --vdb_password, -P     # Database password parameter
9 --vdb_db_name, -d      # Database name parameter
```

A.6.2 Gradio Web Application

The Gradio web interface provides an accessible, interactive platform for exploring the capabilities of the multimodal search engine.

Basic Launch

```
1 python app.py
```

By default, the Gradio application uses configuration from the `.env` file and is accessible at `http://localhost:7860`.

Custom Launch

Environment variables can be overridden at runtime to adapt to different deployment scenarios:

```
1 export VDB_HOST=your_db_host
2 export VDB_PORT=5432
3 export VDB_USERNAME=your_username
4 export VDB_PASSWORD=your_password
5 export VDB_DB_NAME=your_database
6 export GRADIO_USERNAME=your_username
7 export GRADIO_PASSWORD=your_password
8 python app.py
```

This flexibility supports both local development and production deployment.

Web Interface Usage

The web interface is designed for ease of use:

1. Enter a text query or upload an image (or both).
2. Adjust the alpha slider to control the balance between text and image importance.
3. Click "Search for similar products" to initiate the search.
4. Results are displayed in a table, including images, product names, variants, and similarity scores.

This workflow enables intuitive exploration of the system's capabilities, making it accessible to both technical and non-technical users.

A.7 Development Workflow and VSCode Integration

Efficient development and debugging are supported through integration with Visual Studio Code (VSCode), leveraging launch configurations for common tasks.

A.7.1 VSCode Launch Configurations

The project includes several pre-defined launch configurations:

- **Download dataset:** Automates the retrieval of datasets, ensuring consistency and reproducibility.
- **Generate embeddings and populate DB:** Streamlines the embedding generation and database population process.
- **Search test:** Facilitates rapid testing of search functionality with predefined queries.
- **Launch Gradio app:** Simplifies the process of starting the web interface for interactive testing.

A.7.2 Typical Workflow

A recommended development workflow is as follows:

1. Download the desired dataset(s) using the provided scripts or VSCode tasks.
2. Generate embeddings and populate the database.
3. Test search functionality to verify system integrity.
4. Develop and debug new features, leveraging the modular structure and comprehensive test suite.

This workflow supports both iterative development and systematic experimentation.

A.8 Troubleshooting

Robustness and reliability are critical for production systems. The following troubleshooting tips address common issues encountered during setup and operation:

PostgreSQL Connection Error If the application cannot connect to the database, verify that the PostgreSQL service is running and that the pgvector extension is enabled:

Multimodal Search Engine

```
1 brew services list | grep postgresql
2 psql -U postgres -c "SELECT * FROM pg_extension WHERE extname = 'vector';"
```

CUDA Out of Memory Embedding generation can be memory-intensive. If you encounter out-of-memory errors:

- Reduce the batch size in the embedding generation script.
- Switch to CPU processing by setting the device to 'cpu' (at the cost of slower performance).

Model Download Issues If model weights cannot be downloaded:

- Check your internet connection.
- Ensure you have access to the required Hugging Face models.

Gradio Authentication Issues If you cannot log in to the web interface:

- Double-check the .env file for correct credentials.
- Ensure environment variables are loaded correctly in your shell or IDE.

A.9 References

For further technical details, architectural rationale, and experimental results, refer to the main sections of this report, particularly those covering system architecture, implementation, and evaluation. The codebase is extensively documented, and the README provides additional usage examples and troubleshooting guidance.

```
1 {
2   "city": [
3     {
4       "id": 1,
5       "name": "Toronto",
6       "country": "Canada",
7       "population": 6200000
8     },
9     {
10      "id": 2,
11      "name": "New York",
12      "country": "United States of America",
13      "population": 8800000
14    }
15  ]
16 }
```


Reference List

- [1] Charu C. Aggarwal, Alexander Hinneburg, and Daniel A. Keim. “On the Surprising Behavior of Distance Metrics in High Dimensional Spaces”. In: *Database Theory - ICDT 2001, 8th International Conference, London, UK, January 4-6, 2001, Proceedings*. Ed. by Jan Van den Bussche and Victor Vianu. Vol. 1973. Lecture Notes in Computer Science. Springer, 2001, pp. 420–434. DOI: 10.1007/3-540-44503-X_27. URL: https://doi.org/10.1007/3-540-44503-X%5C_27.
- [2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. “Neural Machine Translation by Jointly Learning to Align and Translate”. In: *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*. Ed. by Yoshua Bengio and Yann LeCun. 2015. URL: <http://arxiv.org/abs/1409.0473>.
- [3] Dongmei Chen et al. “Cross-Lingual Passage Re-Ranking With Alignment Augmented Multilingual BERT”. In: *IEEE Access* 8 (2020), pp. 213232–213243. DOI: 10.1109/ACCESS.2020.3041605. URL: <https://doi.org/10.1109/ACCESS.2020.3041605>.
- [4] Ting Chen et al. “A Simple Framework for Contrastive Learning of Visual Representations”. In: *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*. Vol. 119. Proceedings of Machine Learning Research. PMLR, 2020, pp. 1597–1607. URL: <http://proceedings.mlr.press/v119/chen20j.html>.
- [5] E. F. Codd. “A Relational Model of Data for Large Shared Data Banks”. In: *Commun. ACM* 13.6 (1970), pp. 377–387. DOI: 10.1145/362384.362685. URL: <https://doi.org/10.1145/362384.362685>.
- [6] Mayur Datar et al. “Locality-sensitive hashing scheme based on p-stable distributions”. In: *Proceedings of the 20th ACM Symposium on Computational Geometry, Brooklyn, New York, USA, June 8-11, 2004*. Ed. by Jack Snoeyink and Jean-Daniel Boissonnat. ACM, 2004, pp. 253–262. DOI: 10.1145/997817.997857. URL: <https://doi.org/10.1145/997817.997857>.
- [7] Jacob Devlin et al. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*. Ed. by Jill Burstein, Christy Doran, and Thamar Solorio. Association for Computational Linguistics, 2019, pp. 4171–4186. DOI: 10.18653/V1/N19-1423. URL: <https://doi.org/10.18653/v1/n19-1423>.

- [8] Matthijs Douze et al. “The Faiss library”. In: (2024). arXiv: 2401.08281 [cs.LG].
- [9] Yikun Han, Chunjiang Liu, and Pengfei Wang. “A Comprehensive Survey on Vector Database: Storage and Retrieval Technique, Challenge”. In: *CoRR* abs/2310.11703 (2023). DOI: 10.48550/ARXIV.2310.11703. arXiv: 2310.11703. URL: <https://doi.org/10.48550/arXiv.2310.11703>.
- [10] Kaiming He et al. “Momentum Contrast for Unsupervised Visual Representation Learning”. In: *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2020, Seattle, WA, USA, June 13-19, 2020*. Computer Vision Foundation / IEEE, 2020, pp. 9726–9735. DOI: 10.1109/CVPR42600.2020.00975. URL: <https://doi.org/10.1109/CVPR42600.2020.00975>.
- [11] Piotr Indyk and Rajeev Motwani. “Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality”. In: *Proceedings of the Thirtieth Annual ACM Symposium on the Theory of Computing, Dallas, Texas, USA, May 23-26, 1998*. Ed. by Jeffrey Scott Vitter. ACM, 1998, pp. 604–613. DOI: 10.1145/276698.276876. URL: <https://doi.org/10.1145/276698.276876>.
- [12] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. “Product Quantization for Nearest Neighbor Search”. In: *IEEE Trans. Pattern Anal. Mach. Intell.* 33.1 (2011), pp. 117–128. DOI: 10.1109/TPAMI.2010.57. URL: <https://doi.org/10.1109/TPAMI.2010.57>.
- [13] Jeff Johnson, Matthijs Douze, and Hervé Jégou. “Billion-scale similarity search with GPUs”. In: *IEEE Transactions on Big Data* 7.3 (2019), pp. 535–547.
- [14] Karen Spärck Jones. “A statistical interpretation of term specificity and its application in retrieval”. In: *J. Documentation* 60.5 (2004), pp. 493–502. DOI: 10.1108/00220410410560573. URL: <https://doi.org/10.1108/00220410410560573>.
- [15] Vladimir Karpukhin et al. “Dense Passage Retrieval for Open-Domain Question Answering”. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, EMNLP 2020, Online, November 16-20, 2020*. Ed. by Bonnie Webber et al. Association for Computational Linguistics, 2020, pp. 6769–6781. DOI: 10.18653/V1/2020.EMNLP-MAIN.550. URL: <https://doi.org/10.18653/v1/2020.emnlp-main.550>.
- [16] Omar Khattab and Matei Zaharia. “ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT”. In: *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval, SIGIR 2020, Virtual Event, China, July 25-30, 2020*. Ed. by Jimmy X. Huang et al. ACM, 2020, pp. 39–48. DOI: 10.1145/3397271.3401075. URL: <https://doi.org/10.1145/3397271.3401075>.

- [17] Junnan Li et al. “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models”. In: *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*. Ed. by Andreas Krause et al. Vol. 202. Proceedings of Machine Learning Research. PMLR, 2023, pp. 19730–19742. URL: <https://proceedings.mlr.press/v202/li23q.html>.
- [18] Junnan Li et al. “BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation”. In: *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA*. Ed. by Kamalika Chaudhuri et al. Vol. 162. Proceedings of Machine Learning Research. PMLR, 2022, pp. 12888–12900. URL: <https://proceedings.mlr.press/v162/li22n.html>.
- [19] Jiasen Lu et al. “ViLBERT: Pretraining Task-Agnostic Visiolinguistic Representations for Vision-and-Language Tasks”. In: *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*. Ed. by Hanna M. Wallach et al. 2019, pp. 13–23. URL: <https://proceedings.neurips.cc/paper/2019/hash/c74d97b01eae257e44aa9d5bade97baf-Abstract.html>.
- [20] Yury A. Malkov and Dmitry A. Yashunin. “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs”. In: *IEEE Trans. Pattern Anal. Mach. Intell.* 42.4 (2020), pp. 824–836. DOI: 10.1109/TPAMI.2018.2889473. URL: <https://doi.org/10.1109/TPAMI.2018.2889473>.
- [21] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to information retrieval*. Cambridge University Press, 2008. ISBN: 978-0-521-86571-5. DOI: 10.1017/CB09780511809071. URL: <https://nlp.stanford.edu/IR-book/pdf/irbookprint.pdf>.
- [22] Tomás Mikolov et al. “Efficient Estimation of Word Representations in Vector Space”. In: *1st International Conference on Learning Representations, ICLR 2013, Scottsdale, Arizona, USA, May 2-4, 2013, Workshop Track Proceedings*. Ed. by Yoshua Bengio and Yann LeCun. 2013. URL: <http://arxiv.org/abs/1301.3781>.
- [23] Aäron van den Oord, Yazhe Li, and Oriol Vinyals. “Representation Learning with Contrastive Predictive Coding”. In: *CoRR* abs/1807.03748 (2018). arXiv: 1807.03748. URL: <http://arxiv.org/abs/1807.03748>.
- [24] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. “Glove: Global Vectors for Word Representation”. In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, EMNLP 2014, October 25-29, 2014, Doha, Qatar, A meeting of SIGDAT, a Special Interest Group of the ACL*. Ed. by Alessandro Moschitti, Bo Pang,

- and Walter Daelemans. ACL, 2014, pp. 1532–1543. DOI: 10.3115/V1/D14-1162. URL: <https://doi.org/10.3115/v1/d14-1162>.
- [25] Jay M. Ponte and W. Bruce Croft. “A Language Modeling Approach to Information Retrieval”. In: *SIGIR Forum* 51.2 (2017), pp. 202–208. DOI: 10.1145/3130348.3130368. URL: <https://doi.org/10.1145/3130348.3130368>.
- [26] Alec Radford et al. “Learning Transferable Visual Models From Natural Language Supervision”. In: *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*. Ed. by Marina Meila and Tong Zhang. Vol. 139. Proceedings of Machine Learning Research. PMLR, 2021, pp. 8748–8763. URL: <http://proceedings.mlr.press/v139/radford21a.html>.
- [27] S. E. Robertson. “The probability ranking principle in IR”. In: *Readings in Information Retrieval*. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1997, pp. 281–286. ISBN: 1558604545.
- [28] Stephen E. Robertson et al. “Okapi at TREC-3”. In: *Proceedings of The Third Text REtrieval Conference, TREC 1994, Gaithersburg, Maryland, USA, November 2-4, 1994*. Ed. by Donna K. Harman. Vol. 500-225. NIST Special Publication. National Institute of Standards and Technology (NIST), 1994, pp. 109–126. URL: <http://trec.nist.gov/pubs/trec3/papers/city.ps.gz>.
- [29] Gerard Salton, Anita Wong, and Chung-Shu Yang. “A Vector Space Model for Automatic Indexing”. In: *Commun. ACM* 18.11 (1975), pp. 613–620. DOI: 10.1145/361219.361220. URL: <https://doi.org/10.1145/361219.361220>.
- [30] Abigail See et al. “What makes a good conversation? How controllable attributes affect human judgments”. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*. Ed. by Jill Burstein, Christy Doran, and Thamar Solorio. Association for Computational Linguistics, 2019, pp. 1702–1723. DOI: 10.18653/V1/N19-1170. URL: <https://doi.org/10.18653/v1/n19-1170>.
- [31] Hao Tan and Mohit Bansal. “LXMERT: Learning Cross-Modality Encoder Representations from Transformers”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, EMNLP-IJCNLP 2019, Hong Kong, China, November 3-7, 2019*. Ed. by Kentaro Inui et al. Association for Computational Linguistics, 2019, pp. 5099–5110. DOI: 10.18653/V1/D19-1514. URL: <https://doi.org/10.18653/v1/D19-1514>.

- [32] Ashish Vaswani et al. “Attention is All you Need”. In: *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*. Ed. by Isabelle Guyon et al. 2017, pp. 5998–6008. URL: <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845> Abstract.html.
- [33] Xingrui Xie et al. “A Brief Survey of Vector Databases”. In: *2023 9th International Conference on Big Data and Information Analytics (BigDIA)*. 2023, pp. 364–371. DOI: 10 . 1109 / BigDIA60676 . 2023 . 10429609.
- [34] Chengxiang Zhai and John D. Lafferty. “A Study of Smoothing Methods for Language Models Applied to Ad Hoc Information Retrieval”. In: *SIGIR Forum* 51.2 (2017), pp. 268–276. DOI: 10 . 1145 / 3130348 . 3130377. URL: <https://doi.org/10.1145/3130348.3130377>.